MLOPS PIPELINE PACKAGE

FraudShield-v3 Fraud Detection Model

End-to-end MLOps design on Azure: architecture, experiment tracking, CI/CD, drift monitoring, storage optimization and operating procedures for a two-person team

Prepared by	Dennis, MLOps Engineer
Team	MLOps Engineer and Data Scientist (two people, shared on-call)
Platform	Microsoft Azure (East US 2 primary): Azure Machine Learning v2, MLflow, GitHub Actions, Event Hubs, Stream Analytics, Azure Managed Redis, Blob Storage
Version and date	1.0, 11 October 2026
Inputs	Scenario Data Package v1.0; MLflow, GitHub Actions, threshold decision, runbook and diagram templates (all dated 30 July 2026)
Companion files	Dennis_DesignRationale.txt (2–3 sentence rationale); Dennis_MLOpsPipeline_Architecture.drawio (editable Figure 1)

Contents

Design Rationale and Executive Summary 2
1. Assumptions and Constraints 2
2. Architecture Overview 4
2.3 End-to-end architecture diagram 5
3. Experiment Tracking Strategy 8
4. CI/CD Workflow Specification 10
5. Drift Monitoring Design 13
6. Storage Optimization 16
7. Operational Procedures 18
8. Package Self-Check 20
Appendix: Sample Configurations 22
A. MLflow configuration 22
B. GitHub Actions workflows 25
C. Promotion gates 35
D. Drift thresholds and change calendar 36
E. Stream Analytics query 37
F. Deployment and autoscale 38
G. Storage lifecycle and setup 39
H. Runbook command reference 41
I. Configuration consistency matrix 42
J. Template review: corrections log 43
K. References 44

Design Rationale and Executive Summary

> ### Design Rationale
>
> I kept the synchronous scoring path to a single Azure ML managed online endpoint backed by a clustered Redis feature store, and moved telemetry, drift detection, retraining triggers and audit capture onto an asynchronous Event Hubs and Stream Analytics stream, so drift is alerted within two hours without adding latency to the 120 ms budget. Every release passes a pre-traffic load test, a 20% mirrored shadow and a 10% canary on the same endpoint, and the previous model stays warm at 0% traffic, which turns rollback into a traffic switch that automation completes in under five minutes. Gates, thresholds and retention rules live in version-controlled configuration read by both the pipeline and the monitors, so automation does the time-critical work and the two-person team makes the judgment calls.

Executive summary

FraudShield-v3 scores up to 100,000 card transactions an hour, 3.5 times that on Black Friday, against a 120 ms p99 latency target, under PCI DSS v4.0, SOC 2, GDPR and FFIEC obligations, and it is run by two people. The incident history shows where the current setup breaks: drift that went unnoticed for six days ($340K of missed fraud), a rollback that took 23 minutes, a 340% storage bill spike, and a single-node feature store that pushed 8% of transactions past the 200 ms SLA. This package answers each of them with automation rather than with more procedure.

- **Architecture (2).** The gateway calls one managed online endpoint synchronously; features come from Azure Managed Redis with a hard 25 ms timeout. Capture, telemetry, drift and audit are asynchronous. Capacity: 3 instances at peak, 6 on Black Friday.
- **Experiment tracking (3).** One MLflow run contract (20 parameters, 16 metrics, 12 tags) enforced as a gate; lifecycle in registry tags; registered packages copied out so run artifacts can expire at 90 days.
- **CI/CD (4).** One orchestrated release through twelve gates: offline metrics, named approval, pre-traffic load test, mirrored shadow, sample-sized canary. Rollback is a traffic switch to a warm standby in under five minutes.
- **Drift (5).** Hourly, hour-matched windows; PSI, KS and KL calibrated to one effect size; a predicted-fraud-rate ratio and unseen-category rate that flag Incident #1 within an hour. Alerts inside two hours.
- **Storage (6).** Lifecycle rules on every container and an immutable audit account: seven years of explainable decisions for about $270 a month, against about $5,200 if all data stayed Hot.
- **Operations (7).** One automated daily report, P1–P4 severities tied to the same thresholds, and seven procedures either person can run.

Reviewing the supplied templates surfaced 30 defects that would have broken the release chain, the rollback or the compliance case; each is fixed here and logged in Appendix J.

1. Assumptions and Constraints

This section fixes what the design must meet and what it takes as given. Values marked derived were computed from the scenario data; the working is in Sections 2.7 and 5.4.

1.1 Explicit constraints

ID	Constraint (scenario)	Design answer	Section
C1	Peak 100K txn/h (27.8 tps); Black Friday 3.5× (97 tps)	Autoscale 3–6 × Standard_DS3_v2, each tested at ≥ 28 tps; Event Hubs 2 TU auto-inflating to 4; Stream Analytics 3 SU, 6 SU for seasonal events	2.7
C2	p99 ≤ 120 ms per transaction	Latency budget of 72 ms with 48 ms headroom; gate G3 load-tests the whole request path before any live traffic; Redis timeout 25 ms with fallback values	2.6, 4.2

ID	Constraint (scenario)	Design answer	Section
C3	99.9% availability (43.8 min a month)	At least 3 instances, HA Redis, warm standby slot, gateway rules fallback, infrastructure-as-code redeploy to Central US	2, 7
C4	Two people; shared on-call with 4-hour response	Automation performs every action needed within minutes; people approve, investigate and decide	7.1
C5	Automated CI/CD with deployment gates	One orchestrated release, gates G1–G12 in config/gates.yaml	4
C6	Drift detection with numeric thresholds and automated response	PSI, KS, KL, predicted-fraud-rate, unseen-category and labeled-performance thresholds with actions per severity	5
C7	Storage tiering and lifecycle policies	Lifecycle rules on every container; separate immutable audit account	6
C8	Auditable, explainable decisions (SHAP)	Features, score and full SHAP vector of every decision kept 7 years; lineage manifest per model version	3.6, 6
C9	PCI DSS v4.0, SOC 2, GDPR, FFIEC	PAN guard, keyed card hash, private endpoints, OIDC, approval records, EU data stamp	2.2
C10	Rollback < 5 min; drift alert < 4 h	Warm-standby traffic switch (detect ≤ 3 min + switch ≤ 2 min); drift alert ≤ 2 h, Critical ≤ 1 h	4.3, 5.3

1.2 Assumptions

ID	Assumption	Basis	Validation (owner)
A1	Event Hubs Standard with 16 partitions carries peak load	Standard-tier capacity is set by throughput units (1 TU = 1 MB/s or 1,000 events/s in), not per partition; Black Friday ≈ 97 events/s × ~2 KB ≈ 0.2 MB/s per hub	Incoming-bytes and throttling metrics, month 1 (MLOps)
A2	3–6 DS3_v2 instances cover all traffic (derived)	Planned at 70% of the 28 tps floor: 3 at peak, 4 at month-end, 6 on Black Friday (Table 2-4)	G3 load test every release (MLOps)
A3	Feature store is Azure Managed Redis, not Feast	One managed service with an HA replica and no extra control plane for two people to run	Redis latency and fallback alerts (MLOps)
A4	Stream Analytics stays within 6 SU	3 SU baseline, 6 SU scheduled for seasonal events	SU utilisation alert at 80% (MLOps)
A5	Weekly retraining with daily incremental updates	Weekly full retrain is the release unit. Daily: hourly feature materialisation plus a warm-start challenger that is logged and evaluated only; it replaces the weekly candidate if its expected cost beats the champion by ≥ 2%. No daily deployments.	Review after 8 weeks (DS)
A6	Gateway calls scoring synchronously and applies its rules after 250 ms	A decision routed through Event Hubs and Stream Analytics cannot come back within 120 ms	Interface contract with Payments (MLOps)
A7	Fraud labels arrive within 24–72 h	Threshold template; training and labeled monitoring use cohorts at least 72 h old (D-3)	Label completeness in PerfResults_CL (DS)
A8	The card identifier is a keyed hash (HMAC-SHA-256) made upstream	An unkeyed SHA-256 of a 16-digit PAN can be brute-forced; PCI DSS v4.0 requirement 3.5.1.1 expects keyed hashes	Confirm with the QSA (Compliance)
A9	Raw volume 2.1 TB/month (template figure) treated as a ceiling	30.7M transactions/month make that ~68 KB per transaction, far above a typical 1–3 KB event; telemetry ≈ 3 KB/event, audit record ≈ 1 KB in Parquet	Blob inventory after month 1 (MLOps)
A10	EU-resident data is handled by a West Europe stamp	Same Terraform modules and workflows with a region parameter; EU model registered separately; this package details East US 2	DPO sign-off (Compliance)

ID	Assumption	Basis	Validation (owner)
A11	Regional disaster: rebuild in Central US from code within 4 h	Gateway rules fallback covers the gap; 99.9% applies to the in-region service	Annual DR exercise (MLOps)
A12	Low-traffic hours run at ≈ 30% of average (≈ 12,600 txn/h)	Not in the scenario; used only for statistical-power checks in Section 5.4	Hourly volume profile, month 1 (DS)
A13	Loaded cost of team time ≈ $100/h	Used only to price false alarms and retraining reviews	Finance (MLOps)

1.3 Open items

- Azure quota of at least 48 DSv2-family vCPUs in East US 2 for the endpoint: 10 instances at the busiest point of a release (6 live + 2 standby + 2 candidate) plus the 20% Azure ML holds back for upgrades.
- Per-service confirmation that TLS 1.3 is negotiated end to end; some services only expose TLS 1.2 as the configurable minimum.
- Legal view on whether model artifacts trained on EU data may sit outside West Europe (A10) and QSA confirmation of A8.

2. Architecture Overview

2.1 Design principles

- **Small synchronous path:** only the endpoint and Redis sit between the gateway and a decision; telemetry leaves through a background sender.
- **Everything else asynchronous:** Event Hubs carries a copy of each transaction and one telemetry event per score.
- **Automation acts, people decide:** rollback, autoscaling, drift evaluation, retraining and evidence capture are automatic; promotion needs one named approval.
- **One source of truth for numbers:** gates, drift thresholds and the change calendar are version-controlled YAML read by pipelines and monitors.
- **Immutable evidence:** predictions, approvals, drift decisions and rollbacks land in a write-once audit account.

Figure 1 (Section 2.3) shows the East US 2 stamp in four lanes; Tables 2-1 to 2-4 annotate components, interfaces, latency and capacity.

2.2 Security and compliance controls

Requirement	Control in this design
PAN never enters the pipeline (PCI DSS)	Gateway sends a keyed card hash (A8); `score.py` rejects PAN-like fields and Luhn-valid 13–19 digit values (P2, Compliance informed); CI contract test guards the schema.
PII tokenised before storage	No name or address fields in any schema; the schema contract test fails if one is added.
Encryption at rest and in transit	AES-256 storage encryption, customer-managed keys for the audit account; private endpoints for every service; TLS 1.3 check is open item 1.3.
Least privilege and change control (SOC 2)	Managed identities; OIDC from GitHub (no stored secret); traffic-operator role for the function; reviewed PRs, gated release, named approver, CI logs kept 2 years.
Model accountability (FFIEC)	Top-3 reason codes per decision; full SHAP vector and features kept 7 years; lineage manifest per model version.
Residency and minimisation (GDPR)	West Europe stamp for EU data (A10); raw transactions deleted after 2 years; keyed identifiers only.

2.3 End-to-end architecture diagram

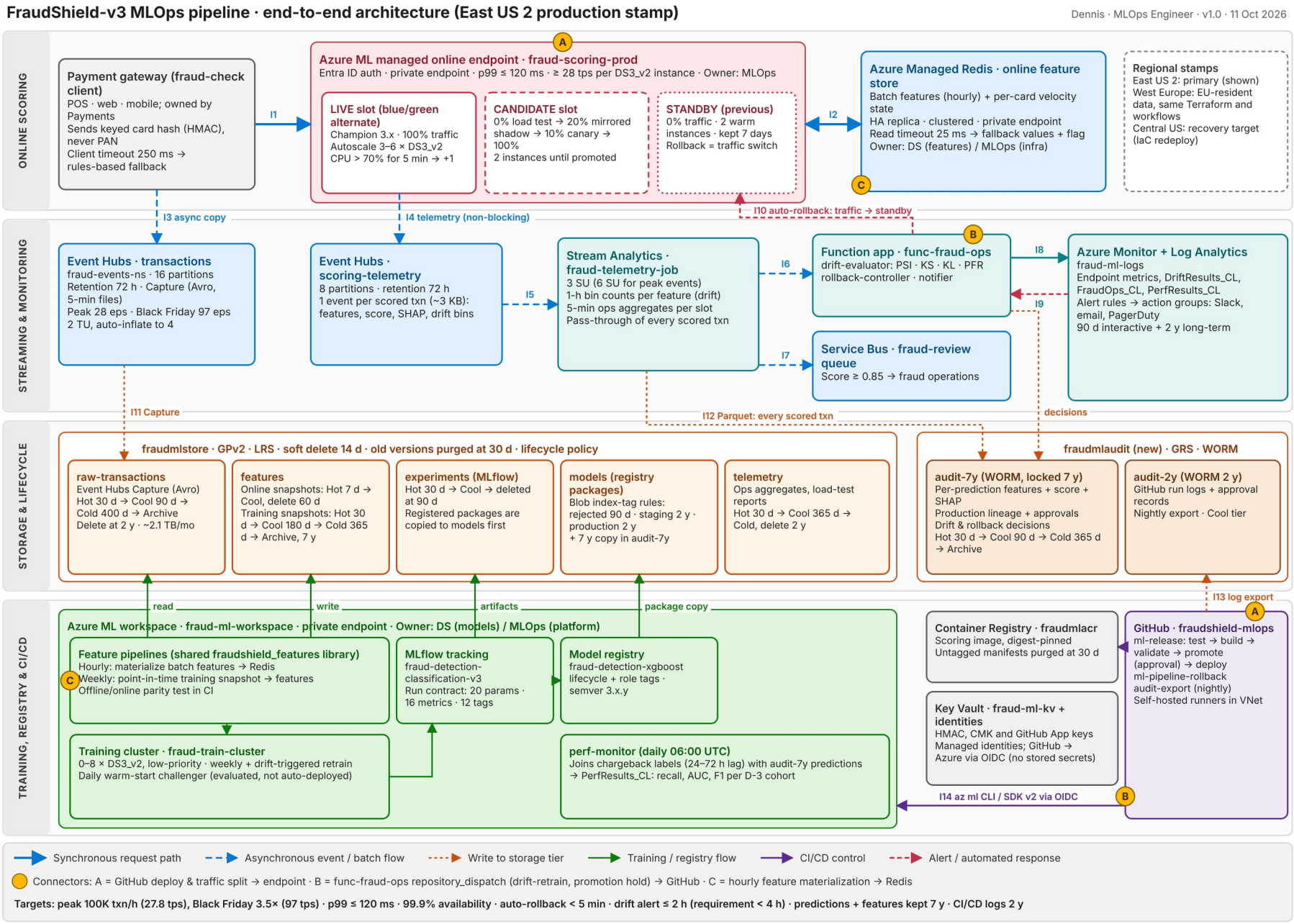


Figure 1. End-to-end architecture, East US 2 production stamp. Interfaces I1–I14 are specified in Table 2-2; badges A–C connect long control links. Editable source: Dennis_MLOpsPipeline_Architecture.drawio.

2.4 Components and responsibilities

Table 2-1. Components, responsibilities, targets and owners

Component	Responsibility	Throughput / latency target	Owner
Payment gateway (external)	Calls scoring synchronously; publishes each transaction to Event Hubs; applies its rules if no answer arrives in 250 ms	250 ms client timeout	Payments team
Managed online endpoint `fraud-scoring-prod`	PAN guard, feature lookup, XGBoost score, decision, top-3 SHAP reason codes, telemetry hand-off. Live, candidate and standby slots	p99 ≤ 120 ms; ≥ 28 tps per instance; 3–6 instances	MLOps
Azure Managed Redis (online store)	Hourly batch features per card hash plus velocity state updated on each request	Read p99 ≤ 15 ms; 25 ms timeout; HA replica	DS (definitions), MLOps (service)
Event Hubs `transactions`	Durable asynchronous copy of transactions; Capture to raw storage	28 events/s peak, 97 on Black Friday; 72 h retention	MLOps
Event Hubs `scoring-telemetry`	One event per scored transaction	≈ 3 KB/event; ≤ 0.3 MB/s on Black Friday	MLOps
Stream Analytics `fraud-telemetry-job`	Hourly drift bins, 5-minute ops aggregates, audit pass-through, review routing	3 SU (6 for events); output lag < 5 min	MLOps
Function app `func-fraud-ops`	drift-evaluator, rollback-controller and notifier; GitHub dispatch and promotion hold	Evaluation ≤ 5 min after window close; traffic switch ≤ 2 min	MLOps
Azure Monitor and Log Analytics	Endpoint metrics, custom tables, alert rules and action groups	Metric alerts evaluated every minute	MLOps
Service Bus `fraud-review`	Hands high-risk transactions (score ≥ 0.85) to fraud operations	Near real time	Fraud operations
Azure ML workspace	Feature pipelines, training cluster (0–8 DS3_v2, low priority), MLflow tracking, model registry, daily perf-monitor job	Weekly training ≤ 2 h; hourly materialisation ≤ 15 min	DS (models), MLOps (platform)
GitHub with self-hosted runners	ml-release, rollback and audit-export workflows; runners inside the VNet reach private endpoints	Release within one working day	MLOps
Storage: `fraudmlstore`, `fraudmlaudit`	Lifecycle-managed operational data; immutable evidence	See Section 6	MLOps

Deployment targets. The online target is the managed endpoint, whose live slot makes every authorisation decision. The streaming target is the candidate slot in shadow mode, which receives a mirrored stream of 20% of live requests so a challenger is scored on real traffic with no customer impact; every scored event then streams through Event Hubs and Stream Analytics for monitoring and audit. A second, asynchronous scorer reading the `transactions` hub was deliberately left out: every transaction is already scored inline, and another scoring path would double what two people must operate.

2.5 Interfaces

Table 2-2. Interfaces shown in Figure 1

ID	From → to	Protocol and authentication	Volume / service level
I1	Gateway → endpoint	HTTPS REST over private endpoint; Microsoft Entra token from the gateway's managed identity	11.7 tps average, 27.8 peak, 97 Black Friday; p99 ≤ 120 ms
I2	Endpoint ↔ Redis	Redis protocol over TLS, Entra authentication; one pipelined read-and-update per transaction	Timeout 25 ms, then fallback values and a flag
I3	Gateway → Event Hubs `transactions`	AMQP over TLS, managed identity	Same rate as I1, ~2 KB/event

ID	From → to	Protocol and authentication	Volume / service level
I4	Endpoint → Event Hubs `scoring-telemetry`	AMQP, background batch sender (≤ 200 ms batches), never on the response path	~3 KB per scored transaction
I5	Event Hubs → Stream Analytics	Event Hub input, dedicated consumer group, managed identity	As I4
I6	Stream Analytics → `func-fraud-ops`	Azure Functions output, batched HTTPS	~300 bin rows per slot per hour; 5-minute aggregates
I7	Stream Analytics → Service Bus	AMQP, managed identity	Transactions with score ≥ 0.85
I8	`func-fraud-ops` → Log Analytics	Logs Ingestion API via data collection rule	DriftResults_CL, FraudOps_CL
I9	Azure Monitor → `func-fraud-ops`	Action-group webhook, common alert schema, Entra-protected	Per alert
I10	`func-fraud-ops` → endpoint control plane	Azure Resource Manager via SDK; custom “endpoint traffic operator” role only	Traffic switch completes in ≤ 2 min
I11	Event Hubs → `raw-transactions`	Event Hubs Capture, Avro files every 5 min or 300 MB	≤ 2.1 TB/month (A9)
I12	Stream Analytics → `audit-7y`	Blob output, Parquet, managed identity	~1 KB per transaction, ~29 GB/month
I13	GitHub → `audit-2y`	Azure CLI with OIDC from a self-hosted runner	Daily export of run logs and approvals
I14	GitHub → Azure ML	az ml CLI v2 over private endpoints; OIDC federated credential, no stored secret	Per release
A/B/C	Connectors	A: GitHub deploy and traffic split → endpoint. B: `func-fraud-ops` → GitHub (repository_dispatch, promotion-hold variable) with a GitHub App token from Key Vault. C: hourly batch write of features to Redis (TTL 35 days)	—

2.6 Latency budget for the synchronous path

The 112 ms p99 in the MLflow template was measured on the model alone. Constraint C2 applies per transaction, so the budget covers the whole request, and gate G3 measures it end to end on the real endpoint before any customer traffic.

Table 2-3. p99 latency budget per transaction

Step	p99 budget
Gateway → endpoint network and TLS (private endpoint, same region)	10 ms
Managed endpoint ingress and routing	10 ms
Request validation and PAN guard	2 ms
Redis feature read and velocity update (one pipelined call; hard timeout 25 ms)	15 ms
Preprocessing (fitted scikit-learn pipeline, 47 features)	8 ms
XGBoost inference (200 trees, depth 8)	10 ms
TreeSHAP contributions for the reason codes (XGBoost `pred_contribs`)	15 ms
Response and hand-off to the background telemetry sender	2 ms
Sum of component p99s	**72 ms**
Headroom for garbage collection, scale-out lag and noisy neighbours	48 ms

Step	p99 budget
Target (C2)	**120 ms**

Two choices make this budget hold. XGBoost computes exact TreeSHAP contributions natively, so explaining each decision costs roughly one more pass over the trees and C8 is met inside the request rather than in a lagging batch job. Incident #4 is handled three ways: a clustered, replicated store; a hard 25 ms timeout with fallback values, so a slow store lowers accuracy rather than raising latency; and a fallback-rate alert (Section 5.3).

2.7 Capacity plan

Table 2-4. Instances needed at 70% of the 28 tps per-instance floor (19.6 tps planned per instance)

Scenario	txn/h	tps	Instances needed	Provisioned (N+1)	Utilisation
Holiday low (0.6 × average)	25,200	7.0	1	3 (minimum)	8%
Average	42,000	11.7	1	3 (minimum)	14%
Peak	100,000	27.8	2	3	33%
Month-end (1.8 × peak)	180,000	50.0	3	4	45%
Black Friday (3.5 × peak)	350,000	97.2	5	6 (pre-scaled to 5)	58%

Latency, not throughput, is the binding constraint. By Little's law the whole endpoint holds about 3 requests in flight at peak and 12 on Black Friday at 120 ms each, so `max_concurrent_requests_per_instance: 10` leaves ample room. Autoscaling adds an instance when CPU stays above 70% for 5 minutes and removes one below 30% for 15 minutes; a Black Friday profile starts the day at 5 of 6 instances (Appendix F.2).

3. Experiment Tracking Strategy

MLflow tracking runs inside the Azure ML workspace, so every training job logs to the same store without extra infrastructure. The strategy has one goal: any production decision can be traced to the exact data, code, parameters and approval behind it, while the experiment store itself stays small.

3.1 Naming conventions

Object	Convention	Example	Rule
Experiment	`fraud-detection-{task}-v{major}`	`fraud-detection-classification-v3`	New major when the feature set, architecture or label definition changes. Monitoring runs use `fraud-detection-monitoring-v3`.
Run	`{model_type}-{config_id}-{yyyymmdd}-{hhmmss}` (UTC)	`xgboost-tuned-optuna-20261011-020412`	config_id from a fixed list: baseline, tuned-optuna, tuned-hyperopt, warmstart, ensemble.
Sweep	Parent run with child trials	`xgboost-tuned-optuna-…` (parent)	Trials are never registered; their artifacts are deleted at 90 days.
Registered model	`fraud-detection-{architecture}`	`fraud-detection-xgboost`	Integer version assigned by Azure ML; semver tag 3.MINOR.PATCH (MINOR = retrain, PATCH = code or threshold change).
Model package	`models/{model}/{semver}/`	`models/fraud-detection-xgboost/3.7.0/`	Copy of the run's artifacts; the registry points here, not at the run.
Training data	`fraud-training-data:{n}`	`fraud-training-data:42`	Immutable Azure ML data asset: 90 days ending D-3.

3.2 What every run must log

The run contract (Appendix A.1) is enforced by `check_run_contract.py` as gate G7, so an incomplete run cannot be promoted.

Group	Count	Contents
Parameters	20	XGBoost and scikit-learn versions; 9 model settings (7 hyperparameters, early stopping, decision threshold); 7 data descriptors (asset version, feature set, window start and end, row counts, positive ratio); global seed; image digest
Metrics	16	AUC; precision and recall at 0.5 and 0.7; F1; FPR; FNR; Brier score; expected cost per 1,000 transactions and the champion's on the same holdout; offline model latency p50/p99; SHAP stability; two fairness gaps
Tags	12	project, model type, pipeline version, git commit, trigger and trigger reference, owner, data source, data region, three compliance flags
Artifacts	7 folders	`model/`, `preprocessing/` (with SHA-256), `shap/`, `drift-baseline/`, `evaluation/`, `environment/`, `lineage/`

Against the template, the TensorFlow seed, epochs and batch size are dropped (meaningless for XGBoost); the Brier score (drift monitoring compares score distributions, so scores must be calibrated) and expected cost (the gate that ties the model to money) are added; the counts now match the checklist.

3.3 Artifact paths

Run artifacts sit in the workspace artifact store (the `experiments` container) under the seven folders above and are deleted after 90 days. When a candidate passes the offline gates, `package_and_register.sh` copies the run to `models/fraud-detection-xgboost/<semver>/`, tags the blobs for retention, and registers that copy. When a version reaches production, the package plus its lineage manifest, validation report and approval record are also written to `audit-7y/models/` in the immutable audit account. Cleaning the experiment store therefore never breaks a registered model: Incident #3 cannot recur, and lineage survives.

3.4 Model registry tagging and lifecycle

MLflow stage transitions are deprecated upstream (since MLflow 2.9), so lifecycle is recorded as tags that `az ml` and the MLflow client query the same way. All values are lowercase, which removes the template's `staging`/`Staging` mismatch.

lifecycle tag	Entered when	Set by	Package retention
staging	All offline gates pass	ml-pipeline-validate	2 years
rejected	Any offline gate fails	ml-pipeline-validate	90 days (debugging)
approved	G8 approval; `approved_by` and `approved_at` recorded	ml-pipeline-promote	—
production, role = champion	Full traffic after canary; `stability=passed` after 24 h (G12)	ml-pipeline-deploy	2 years working copy + 7 years audit copy
standby, role = previous	Replaced by a new champion; kept warm for 7 days	ml-pipeline-deploy	As production
archived	Standby slot reused by the next release	ml-pipeline-deploy	As production
rolled-back	Traffic removed by a rollback	ml-pipeline-rollback	As production (evidence)

The template's auto-promotion after four hours without objection is removed: with a four-hour on-call SLA it amounts to no approval, and it breaks approval traceability.

3.5 Evaluation gating

Table 3-1. Offline gates evaluated in ml-pipeline-validate on the 30-day holdout

Gate	Check	Threshold	Why
G1	AUC-ROC	≥ 0.94	Scenario performance baseline
G2	F1; precision and recall at 0.5	≥ 0.86; ≥ 0.82; ≥ 0.91	Scenario baseline (the template gated only F1)
G4	False-positive rate	≤ 0.02	Each blocked legitimate payment costs $8 in friction
G5	SHAP stability: mean Spearman correlation of global importance across 5 bootstrap resamples	≥ 0.85	Reason codes given to customers and examiners must not change with sampling noise
G6	Flag-rate gap and equalised-odds gap across Compliance-approved segments (country, channel, amount band)	≤ 0.05 each, and ≤ 0.01 worse than the champion	Consistent treatment; protected attributes are never model inputs
G7	Run contract complete; upstream test, schema, PAN-guard and parity checks green	Pass	Auditability
G9	Expected cost per 1,000 transactions = 127 × FN + 8 × FP, on the same holdout as the champion	≤ champion	A model can pass G1–G4 and still lose money

G9 also governs the decision threshold. With calibrated scores the cost-minimising cut-off is c_FP / (c_FP + c_FN) = 8 / 135 ≈ 0.06, far below today's 0.5, so the threshold is a versioned model parameter owned by the Data Scientist. It changes only through a PATCH release in which G9 shows the saving and G4 caps the customer friction.

3.6 Reproducibility and lineage

- **Environment:** conda file, hashed requirements lock and the image digest are logged; deployments pin the image by digest.
- **Randomness:** one global seed (42) for NumPy, Python and XGBoost.
- **Data:** immutable Azure ML data asset version and the SHA-256 of the fitted preprocessing pipeline.
- **Chain of custody:** each telemetry record carries `model_version`; the registry version carries `source_run_id`, `data.version`, approver and deployment time; `lineage.json` links data asset, feature pipeline commit, training job and sweep. Training snapshots are kept seven years, so any decision from the last seven years can be traced and reproduced.

4. CI/CD Workflow Specification

All automation runs in GitHub Actions (`fintech-corp/fraudshield-mlops`). The template's six workflows remain, but one orchestrator calls them instead of chaining through `workflow_run`: GitHub stops such chains after three levels, so the template's Deploy would never have fired, and artifacts could not cross runs without run IDs. With reusable workflows and `needs:`, one run carries one candidate from test to production.

4.1 Workflow inventory

Workflow	Trigger	Runner	Purpose and outputs
`ml-release.yml`	Weekly cron (Sunday 02:00 UTC); `repository_dispatch: drift-retrain`; manual with a reason	—	Orchestrator: calls the five stages below with `needs:`; one release at a time (concurrency group)
`ml-pipeline-test.yml`	Pull request to main; push to develop; called by release	GitHub-hosted	Lint; unit tests with 85% coverage; schema contracts; PAN guard; offline/online feature parity; config consistency; Terraform validate

Workflow	Trigger	Runner	Purpose and outputs
`ml-pipeline-build.yml`	Called	Self-hosted in VNet	Scoring image to ACR (digest); Azure ML training pipeline; outputs run ID, data version, window end
`ml-pipeline-validate.yml`	Called	Self-hosted	G7 and offline gates; packages and registers as staging or rejected; validation report
`ml-pipeline-promote.yml`	Called	Hosted and self-hosted	Promotion-hold check; G8 approval; approver written to registry and audit account
`ml-pipeline-deploy.yml`	Called	Self-hosted	G3 load test, G10 shadow, G11 canary, full promotion, warm standby, audit copy
`ml-pipeline-rollback.yml`	Manual (reason required); `repository_dispatch: auto-rollback-triggered`	Self-hosted	Traffic to rollback target, capacity restore, verification, audit record, notifications
`audit-export.yml`	Daily 03:15 UTC	Self-hosted	Copies run logs and approval history to `audit-2y`; GitHub keeps logs for at most 400 days

4.2 Release flow and gating conditions

Table 4-1. Gating conditions by stage

Stage	Pass condition	On failure
Test	All checks green	Pull request blocked or release stopped
Build	Azure ML pipeline completes within 150 min; training run ID resolved	Release stopped; Slack notification
Validate (G7, G1, G2, G4, G5, G6, G9)	Run contract complete and every offline gate passes (Table 3-1)	Registered as rejected (kept 90 days); report posted
Promote (G8)	No promotion hold covers the candidate's data window; one approval from a reviewer who did not trigger the run	Release stopped; hold message names the retrain needed
Deploy: G3 load test	Candidate slot at 0% traffic sustains 28 tps per instance for 10 min with p99 ≤ 120 ms and no errors; requests are synthetic and header-routed	Candidate deleted; live slot untouched
Deploy: G10 shadow	20% of live requests mirrored for 30 min: predicted-fraud-rate ratio 0.80–1.25 and score PSI < 0.10 against live; no errors	Mirror off; release aborted
Deploy: G11 canary	10% of live traffic until ≥ 5,000 requests (cap 180 min): p99 ≤ 120 ms, error rate ≤ 0.1%, ratio 0.80–1.25; abort rules checked every 60 s	Traffic back to the live slot within one poll
Post-release: G12	24 h without a P1/P2 incident or rollback; then `stability=passed`	Automatic rollback to the warm standby

The canary is sized by requests, not minutes. A 30-minute canary at 10% sees about 2,100 requests at average traffic, and far fewer in the 02:00–06:00 UTC window the runbook template preferred: too few for a p99 or a 0.1% error rate (two errors would fail it). 5,000 requests (about 70 minutes at average traffic) give about 50 tail samples and make 0.1% equal to five errors. Releases therefore run Tuesday–Thursday, 13:00–17:00 UTC, between the daily peaks and while both people are online, outside the freezes in Appendix D.2.

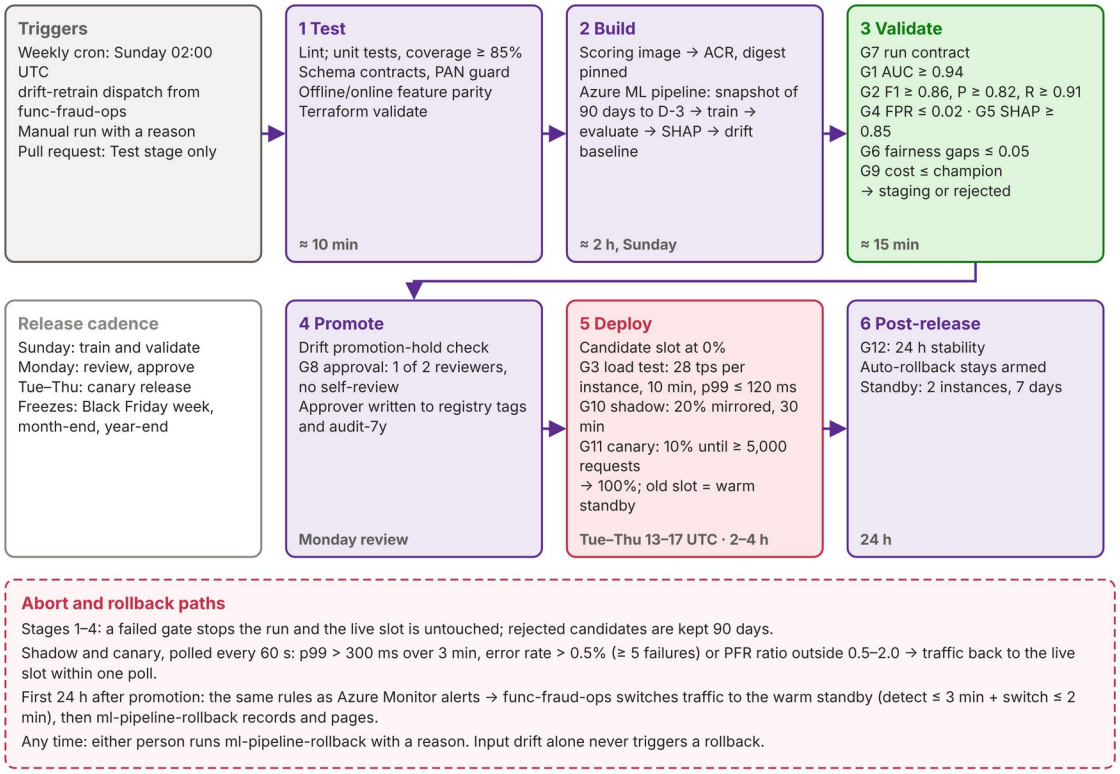


Figure 2. One ml-release run per candidate. Every threshold shown is read from config/gates.yaml (Appendix C).

Key step from `ml-pipeline-deploy.yml` (full file in Appendix B.6):

```
      - name: G11 - canary - 10 % live traffic until >= 5,000 requests
        run: |
          az ml online-endpoint update -n $EP \
             --traffic "${{ steps.slots.outputs.live }}=90 ${{ steps.slots.outputs.target }}=10" \
             --set tags.release_state=canary
          # On an abort rule the controller itself restores live=100 within one poll, then exits 1.
          python scripts/release_controller.py observe --stage canary --gates config/gates.yaml \
                 --live ${{ steps.slots.outputs.live }} --candidate ${{ steps.slots.outputs.target }}
```

4.3 Rollback design

The previous version stays deployed at 0% traffic with two instances for seven days, so rollback never waits for a new deployment to provision. The endpoint tag `rollback_target` always names the slot to return to: the live slot during shadow and canary, the previous champion after promotion.

Table 4-2. Rollback timing

Step	Automatic path	Manual path
Detect	Azure Monitor metric alert evaluated every minute over a 3-minute window: p99 > 300 ms, or error rate > 0.5% with at least 5 failures	On-call person decides on a P1 or P2 (Section 7.3)
Trigger	Action group → `func-fraud-ops` rollback-controller	`gh workflow run ml-pipeline-rollback.yml -f reason=…`
Act	Traffic update to `rollback_target`, already warm: about 1–2 min	Same switch through the workflow: about 2–3 min including runner start
Record	Function dispatches the rollback workflow: audit record, registry tag, Slack, PagerDuty	Same

Step	Automatic path	Manual path
Total	≤ 5 min from breach to recovery	≤ 5 min from decision to recovery

Against Incident #2: the v2.4 bug (890 ms p99) would have failed G3 before any customer traffic; had it slipped through, the canary abort would have exposed about 210 transactions (10% of three minutes at average volume) instead of 4,200. Automatic rollbacks are pre-approved by written policy RB-1, only during shadow, canary and the 24 hours after a release and only on the rules in `gates.yaml`; each record names the rule and alert ID, and manual rollbacks record the person and reason, which gives the required approval traceability.

4.4 Environments, identity and runners

- **GitHub environments:** `production` requires one of two reviewers (Data Scientist, MLOps Engineer) with "prevent self-review" on and main-branch only; `production-rollback` has no reviewers (policy RB-1) and allows main and hotfix branches.
- **Identity:** OIDC federated credentials for Azure; only client, tenant and subscription IDs are stored, as repository variables. Secrets: `SLACK_WEBHOOK_URL`, `PAGERDUTY_ROUTING_KEY`. The GitHub App key used by `func-fraud-ops` lives in Key Vault.
- **Runners:** two ephemeral self-hosted runners (Standard_D2s_v5) in the training subnet for every job that touches Azure, because the workspace, ACR and storage sit behind private endpoints; jobs with no Azure access use GitHub-hosted runners.
- **Concurrency:** releases are serialised in one group; rollback has its own group, so it is never queued behind a release.

5. Drift Monitoring Design

Incident #1 is the requirement in its sharpest form: a new partner's merchant categories shifted the input distribution, the predicted fraud rate fell from 2.3% to 0.8%, and nobody noticed for six days. The design below detects that pattern within an hour, triggers a retrain automatically and never needs a person to be watching a dashboard.

5.1 Telemetry metrics

Signal	Source	Granularity	Used for
Per-request record: request ID, time, keyed card hash, slot, model version, mirror and synthetic flags, 47 features, score, decision, threshold, 47 SHAP values, top-3 reason codes, fallback and unseen-category flags, server latency, bin indices for 12 monitored features	`score.py` → Event Hubs	Every transaction	Audit (7 years), drift, operations
Endpoint metrics: request latency p50/p95/p99, requests per minute, 4xx/5xx, CPU and memory per slot, throttling	Azure ML → Azure Monitor	1 minute	Latency and error alerts, autoscale, canary abort
Ops aggregates: count, flags, predicted fraud rate (PFR), fallback rate, unseen-category rate, server p99, per slot and mirror flag	Stream Analytics → FraudOps_CL	5 minutes	Shadow and canary comparison, ops alerts
Drift aggregates: bin counts per feature and for the score; attribution mass per feature	Stream Analytics → function → DriftResults_CL	1 hour	PSI, KS, KL
Labeled performance: recall, precision, AUC, F1, label completeness	perf-monitor → PerfResults_CL	Daily, D-3 cohort	Performance drift
Platform health: Redis latency and memory, Event Hubs throttling, Stream Analytics SU% and watermark delay, function failures	Azure Monitor	1–5 minutes	P2/P3 platform alerts

5.2 Streaming and aggregation architecture

1. `score.py` assigns each monitored value to a bin of the serving model's own baseline (20 equal-frequency bins per numeric feature; category buckets plus an "unseen" bucket) and hands the telemetry event to the background sender.
2. Stream Analytics groups events in tumbling windows: one hour per slot, feature and bin (with hour of day and weekday or weekend), and five minutes for operational aggregates. It also writes every record to `audit-7y` and routes scores ≥ 0.85 to fraud operations.
3. `func-fraud-ops` loads the matching baseline profile (48 profiles: hour of day × weekday/weekend), computes PSI and KL on deciles, KS on the 20-bin CDF, KL on the score and attribution distributions, and the PFR ratio against the hour-of-week expectation, then writes values and severity to DriftResults_CL within five minutes of the window closing.
4. The function performs the automated actions itself (promotion hold, retrain dispatch), so they do not depend on the alert path. Azure Monitor log alerts on DriftResults_CL handle notification.
5. Baselines change only when the model changes, because they are an artifact of the model version (Section 3.2).

Hourly, hour-matched windows replace the template's five-minute windows for a concrete reason: inside five minutes hour_of_day and day_of_week are constant, so against an all-day baseline their PSI is huge in every window, and the transaction mix also varies by hour. Matching the baseline to the hour removes expected variation, so what remains is drift. Time features are excluded from drift scoring; five-minute windows are kept for operational signals.

5.3 Drift metrics and thresholds

Table 5-1. Thresholds (config/drift_thresholds.yaml). Medium and High need two consecutive windows; Critical fires on the first

Signal	Applied to	Window	Low	Medium	High	Critical
PSI (10 deciles)	12 monitored features and the score	1 h	≥ 0.10	≥ 0.20	≥ 0.25	≥ 0.25 on 3+ features and score KL ≥ 0.125
KS statistic D (α = 0.05/8)	8 numeric features	1 h	≥ 0.10	≥ 0.15	≥ 0.20	—
KL divergence	Score distribution (20 bins); SHAP attribution shares	1 h	≥ 0.05	≥ 0.10	≥ 0.125	See PSI
PFR ratio (observed / expected flag rate)	Live decisions	1 h	outside 0.85–1.18	outside 0.75–1.33	outside 0.70–1.43	outside 0.50–2.00
Unseen merchant-category rate	Merchant category feature	1 h	≥ 0.5%	≥ 1%	≥ 3%	≥ 5%
Feature fallback rate	Redis timeouts	5 min	≥ 0.5%	≥ 1%	≥ 5%	—
Recall drop vs champion baseline	Labeled D-3 cohort	Daily	≥ 0.01	≥ 0.02	≥ 0.03	≥ 0.05
AUC drop / F1 drop	Labeled D-3 cohort	Daily	≥ 0.01 / 0.02	≥ 0.02 / 0.04	≥ 0.03 / 0.06	—

Monitored features: the eight numeric ones (amount_log, avg_txn_amount_30d, txn_count_24h, unique_merchants_7d, time_since_last_txn_minutes, amount_vs_avg_ratio, velocity_score_1h, device_risk_score) and four categorical ones (merchantCategory_encoded, distance_flag, isOnline, geo_anomaly_flag). A High alert therefore fires at most two hours and five minutes after onset and a Critical within one hour, both inside the four-hour requirement with room for a missed window. Expected PFR for each hour of the week is the trailing four-week median for that hour, reset after each release from the canary measurement.

5.4 Why these thresholds fit the business risk and data volume

Consistent effect size across the three tests

PSI, KL and KS measure different things, so the same number means a different amount of drift for each. I calibrated them on a simulated shift in the mean of a normally distributed feature (400,000-row baseline, 10 deciles):

Mean shift (standard deviations)	0.25	0.32	0.40	0.46	0.52	0.60
PSI	0.06	0.10	0.16	0.21	0.26	0.35
KL	0.03	0.05	0.08	0.10	0.13	0.17
KS D	0.10	0.13	0.16	0.19	0.21	0.24

The High thresholds (PSI 0.25, KL 0.125, KS 0.20) all correspond to about half a standard deviation, so the three tests corroborate each other instead of firing at different times. The template's KL 0.15 sits near 0.55σ and its KS 0.15 near 0.4σ, so its combined rule (PSI ≥ 0.25 and KL ≥ 0.15 and D ≥ 0.15) mixed three different drift sizes.

Data volume and statistical power

Quantity	Low-traffic hour (≈ 12,600)	Average hour (42,000)	Peak hour (100,000)
PSI noise floor with no drift (10 bins)	0.0008	0.0003	0.0002
KS critical D at α = 0.05/8	0.016	0.010	0.008
Expected flags per hour at PFR 2.53% (± 1 SD)	319 (± 5.6%)	1,062 (± 3.1%)	2,529 (± 2.0%)
A 30% PFR drop (High band) in standard deviations	5.4	9.8	15

At this volume significance is not the constraint: with 8,333 transactions (the template's five-minute window) even D = 0.05 has $p \approx 5 \times 10^{-17}$, so the template's p-value bands could not separate its alert levels. Thresholds are therefore effect sizes, with the Bonferroni-corrected KS test kept only as a guard for thin windows. Noise cannot reach even the Low thresholds, so realistic false alarms are real but harmless shifts such as seasonal events, which persistence and the event calendar handle. Labels have the same power: about 23,000 confirmed frauds a day make a 0.01 recall drop about 5.5 standard errors in a single daily cohort.

Business risk

Event	Cost per hour (average / peak)	Basis
1 percentage-point recall loss	$1,227 / $2,921	966 or 2,300 frauds an hour × 1% × $127 (the scenario rounds the average to $1,270)
3-point recall loss (High)	$3,680 / $8,763	As above × 3
Scoring slower than 200 ms	$3,200	Scenario cost model
Service downtime	$45,000	Scenario cost model
Drift-triggered retrain (one-off)	≈ $250	About 8 low-priority DS3_v2 node-hours (a few dollars) plus about 2.5 h of review at the assumed $100/h
Non-actionable page (one-off)	≈ $100 plus attention	About an hour of one person (A13)

Two conclusions follow. Retraining is cheap and drift is expensive, so a challenger retrain starts automatically at Medium: even if only one Medium alert in twenty were a real one-point recall loss lasting a day, the expected saving would be about $1,470 per alert against about $250 per retrain. Attention is the scarce resource for two people, so pages start only at High, with a budget of at most two non-actionable pages a month. With alerts inside two hours, a three-point recall loss costs at most about $7,400 at average volume before anyone is paged, against $340K in Incident #1.

5.5 Alerting and automated responses

Severity	Channel	Automated action	Human response
Low	Daily digest in Slack	Logged in DriftResults_CL	Next business day
Medium	Slack, email, Jira ticket	Challenger retrain via `repository_dispatch` (at most one per 24 h)	Within 4 h, business hours
High	PagerDuty to the on-call person, Slack	Promotion hold and challenger retrain	1 h in business hours; 4 h out of hours
Critical	PagerDuty to both people	Hold, retrain, fraud operations notified; rollback only if within 24 h of a release	15 min in business hours; 4 h out of hours (automation has already acted)

5.6 Retraining triggers and guardrails

- **T1 scheduled:** weekly, Sunday 02:00 UTC, on 90 days of data ending D-3.
- **T2 distribution drift:** any signal at Medium or above for two consecutive hours, or unseen-category rate ≥ 1%.
- **T3 labeled performance:** recall or AUC drop of 0.02 or more.
- **T4 manual:** the Data Scientist, for example after updating merchant-category encodings.
- **Promotion hold:** on High or Critical, `func-fraud-ops` sets `PROMOTION_HOLD_SINCE` to the drift onset. The promote stage then blocks any candidate whose training window ends before that date, so a model trained on the old population cannot ship into the new one, while the drift-triggered challenger can still go through with approval.
- **Seasonal events:** during the events in Appendix D.2, distribution-only signals drop one level and do not retrain automatically, because a model retrained on three days of Black Friday traffic would be worse afterwards. PFR ratio, unseen categories and labeled performance keep full severity.
- **No rollback on input drift:** live and candidate slots see the same inputs, so rolling back cannot fix input drift. During shadow and canary only divergence between the slots' outputs aborts a release.

5.7 Incident #1 replayed against this design

The new partner's merchant categories map to the "unseen" bucket, so within the first hour the unseen-category rate passes 5% (Critical) and the PFR ratio, at 0.8% / 2.3% = 0.35, is far outside 0.50–2.00 (Critical). `func-fraud-ops` pages both people, sets the promotion hold, notifies fraud operations to apply temporary rules for the new merchants, and dispatches a challenger retrain. The Data Scientist adds the new encodings, and the first retrain that includes labeled post-onset data clears the hold. Exposure shrinks from six days to about an hour plus the time for fraud-operations rules, against $340K in the original incident.

6. Storage Optimization

Incident #3 came from data with no owner and no expiry. Here every class of data has a container, a tier schedule and a deletion age enforced by lifecycle policies (Appendix G), and seven-year evidence lives in a separate immutable account, so the operational account can be cleaned aggressively.

6.1 Tiers and retention

Table 6-1. Tier schedule by data class (ages since last modification)

Data and location	Hot	Cool	Cold	Archive	Deleted	Why
Raw transactions (Capture Avro) — `fraudmlstore/raw-transactions`	0–30 d	31–90 d	91–400 d	401–730 d	2 y	90 days covers retraining and backfills; Cold keeps last year's Black Friday online for seasonal training; the payments platform is the system of record

Data and location	Hot	Cool	Cold	Archive	Deleted	Why
Online feature snapshots — `features/online-snapshots`	0–7 d	8–60 d	—	—	60 d	Only needed to rebuild Redis
Training snapshots (data assets) — `features/training-snapshots`	0–30 d	31–180 d	181–365 d	1–7 y	7 y	Lineage for any production model (C8)
MLflow run artifacts — `experiments`	0–30 d	31–90 d	—	—	90 d	Incident #3; registered packages are copied out first
Model packages — `models` (blob index tag `retention`)	30 d or 180 d	to 2 y	—	—	rejected 90 d; others 2 y	Hot 30 d for staging, 180 d for production; working copies for redeploy; 7-year copy in audit-7y
Ops aggregates, load-test reports — `telemetry`	0–30 d	31–365 d	1–2 y	—	2 y	Trend analysis
Per-prediction features, score, SHAP — `audit-7y/predictions`	0–30 d	31–90 d	91–365 d	1–7 y	after 7 y	7-year audit retention; immutable
Lineage, approvals, drift and rollback decisions — `audit-7y`	0–30 d	31 d–2 y	—	2–7 y	after 7 y	Audit trail
CI/CD run logs and approvals — `audit-2y`	0–1 d	1 d–2 y	—	—	after 2 y	SOC 2 evidence; GitHub keeps logs ≤ 400 days

Outside Blob Storage: Log Analytics keeps 90 days interactive and two years in total for the MLOps tables; Event Hubs keeps 72 hours (up from 24, so a weekend outage of Capture or a consumer loses nothing under a four-hour on-call SLA); Redis keys expire after 35 days (batch features) or 25 hours (velocity state); ACR purges untagged manifests after 30 days.

6.2 Archival steps and mechanics

- **Age-based rules** run daily on last-modified age. Every schedule respects the minimum stay of each tier (30 days in Cool, 90 in Cold, 180 in Archive), so no early-deletion charges arise.
- **State-based rules** for model packages use blob index tags set by the workflows, because a package's fate depends on its lifecycle state, not its age.
- **Versioning and soft delete** (14 days) protect `fraudmlstore` from mistakes, and a clean-up rule deletes previous versions after 30 days. Versioning without that rule would quietly recreate Incident #3.
- **Immutability:** `fraudmlaudit` uses container-level time-based retention (7 years and 2 years), left unlocked for a 30-day validation period and then locked. The account is geo-redundant (GRS), so a copy exists in the paired region. Archive tiering works on immutable blobs; deletion becomes possible only after expiry.
- **Retrieval:** evidence older than a year is archived and needs rehydration, which can take up to 15 hours at standard priority, so the audit procedure (R7) starts rehydration first.

6.3 Sizing and cost

Table 6-2. Steady state after seven years, tiered versus everything kept in Hot

Data class	Ingest per month	Stored (tiered)	$ per month (tiered)	Stored (all Hot, never deleted)	$ per month (all Hot)
Raw transactions	2.1 TB	49.2 TB	$233	172.3 TB	$4,057
Online feature snapshots	150 GB	0.3 TB	$4	12.3 TB	$290
Training snapshots	30 GB	2.5 TB	$8	2.5 TB	$58
Experiment artifacts	400 GB	1.2 TB	$19	32.8 TB	$773
Audit predictions	29 GB	2.4 TB	$7	2.4 TB	$56
Models, telemetry, decisions, CI logs	≈ 2 GB	0.1 TB	< $1	0.2 TB	$4

Data class	Ingest per month	Stored (tiered)	$ per month (tiered)	Stored (all Hot, never deleted)	$ per month (all Hot)
Total		**55.6 TB**	**≈ $271**	**222.4 TB**	**≈ $5,238**

Planning prices: Hot $0.023, Cool $0.013, Cold $0.004 per GB-month (East US 2 list prices published February 2025), Archive $0.002 assumed; transactions, rehydration and the GRS premium are excluded, so the figures compare designs rather than set a budget. The experiment store is bounded near 1.2 TB (Incident #3 had accumulated 4.2 TB), and the tiered cost is flat after year two while the untiered cost keeps climbing: $264 against $1,497 a month at year two, $271 against $5,238 at year seven.

6.4 Cost guardrails

- An Azure Cost Management budget on storage with alerts at 80% and 100%, routed to Slack.
- A daily blob inventory report; the daily health check flags any container growing more than 20% above forecast (P3).
- Two-year retention in Log Analytics applies only to the four MLOps tables; everything else keeps the workspace default.

7. Operational Procedures

7.1 Operating model for two people

Either person can run every procedure: the MLOps Engineer leads on platform and incidents, the Data Scientist on model decisions, and each covers the other. On-call rotates weekly; out of hours the on-call person responds within four hours, which is safe because nothing that must happen within minutes waits for a person (autoscaling, rollback, gateway fallback and drift evaluation are automatic). All changes go through pull requests; the only console action allowed is the break-glass rollback in Appendix H, followed by the workflow that records it.

When	Task	Who	Effort
Daily, 07:00 UTC	Read and acknowledge the automated health report (7.2)	On-call person	10 min
Monday	Review Sunday's candidate validation report; approve or reject (G8)	Data Scientist (MLOps if away)	30 min
Tuesday–Thursday, 13:00–17:00 UTC	Watch the automated canary; act only on an abort	Release owner	≈ 20 min attention
Friday	Confirm 24 h stability tag and standby slot; review false alarms	Either	15 min
Monthly, first Monday	Threshold review (false-alarm rate, performance trend), cost review, access review, restore test of one archived audit blob	Both	1 h
Quarterly	Rollback drill in a deploy window, audit-evidence dry run, Central US redeploy test	Both	Half a day

7.2 Daily health check

A scheduled job posts one report to Slack at 07:00 UTC, replacing the template's seven manual checks; the on-call person acknowledges it in the thread and notes any action. Reports are kept in `telemetry/daily-health/` for two years. It covers:

- Endpoint state, traffic split, release state and rollback target; instance counts per slot.
- Last 24 h: p99 latency, error rate, peak CPU, throttling; traffic against the same day last week.
- Drift: highest PSI, KS and KL with feature names, PFR ratio range, unseen-category and fallback rates, any Medium or worse alert and its action.
- Labeled performance for the D-3 cohort against the champion baseline, with label completeness.

- Storage growth against forecast, audit-export success, CI/CD failures on main. Anything red goes to triage (7.3).

7.3 Incident severity and triage

Severity	Triggers	Response
P1 Critical	Endpoint not serving for 2 min or provisioning failed; p99 > 300 ms for 5 min; error rate > 0.5% for 5 min; Critical drift; any executed rollback	Page both. If a release happened in the last 7 days and evidence points to it, run R2. Otherwise check Azure status and R5. 15 min in business hours, 4 h out of hours (the gateway fallback and automation are already acting).
P2 High	p99 > 200 ms for 15 min; error rate > 0.1% for 15 min; High drift; autoscale at maximum for 15 min or throttling; release aborted; PAN guard rejection; audit export failed twice	Page on-call. Drift → R3; capacity → R5 or R6; PAN guard → Compliance. Within 1 h in business hours.
P3 Medium	p99 120–200 ms for 30 min; Medium drift; fallback rate > 1%; storage growth > 20% above forecast	Slack and Jira; handle the same business day.
P4 Low	Anything else in a warning band	Daily digest; next business day.

Two rules keep triage simple: if scoring is down, the gateway is already applying its rules, so the only question is whether the last release (roll back) or the platform (R5, Azure status) caused it; if it is input drift, never roll back, retrain (R3).

7.4 Runbook procedures

Procedure	Trigger	Steps	Done when
R1 Verify an automatic rollback	PagerDuty “Rollback executed”	1. Open the rollback workflow run; confirm traffic is on `rollback_target`. 2. Check p99 and errors for the last 10 min (Appendix H). 3. Confirm the audit record exists in `audit-7y/rollbacks/`. 4. Open a post-mortem ticket within 24 h; fix forward through a normal release.	Healthy for 30 min; record present (≤ 15 min of work)
R2 Manual rollback	P1 or P2 with a release in the last 7 days	1. `gh workflow run ml-pipeline-rollback.yml -f reason="INC-…"`. 2. Watch the run (about 3 min). 3. Verify as R1. 4. If GitHub is down, use the break-glass CLI in Appendix H, then run the workflow to write the record.	Traffic switched in ≤ 5 min
R3 Drift response and retraining	Medium or worse drift; performance drop	1. Read DriftResults_CL: which signal and features; seasonal event? 2. Unseen categories: the DS updates the encoding map by pull request. 3. Confirm the challenger run started (trigger=drift) or start one. 4. Approve only a candidate whose data includes post-onset days. 5. Release in the next window; Critical may use the emergency override. 6. Clear the hold after 24 h stable; log the action.	Candidate ready ≤ 8 h after a High alert
R4 Weekly release	Sunday schedule	1. Sunday: training and validation run automatically. 2. Monday: review the report; approve or reject. 3. Tuesday–Thursday window: deploy runs; watch the canary. 4. Friday: confirm the stability tag.	Champion promoted and stable 24 h

Procedure	Trigger	Steps	Done when
R5 Feature store degradation	Fallback rate > 1% or Redis latency alert	1. Check Redis server load, memory and connections. 2. Check the hourly materialisation job. 3. Scale Redis up through a Terraform pull request. 4. If fallback stays > 5% for 30 min, tell fraud operations accuracy is reduced.	Fallback < 0.5% for 1 h
R6 Seasonal peak preparation	Two weeks before Black Friday or a known event	1. Confirm the autoscale event profile (minimum 5) and vCPU quota. 2. Schedule Stream Analytics at 6 SU and Event Hubs auto-inflate to 4 TU. 3. Load-test Redis at 100 tps. 4. Confirm the change freeze; include last year's event in the training window.	Checklist signed off by both
R7 Audit evidence request	Examiner or customer request	1. Start rehydration of archived blobs first. 2. From the transaction: model version, score, SHAP and reason codes in `audit-7y/predictions`. 3. From the version: package, lineage, validation report and approver in `audit-7y/models`. 4. CI/CD logs from `audit-2y`.	Evidence pack within 5 business days

7.5 Escalation

Level	Who	When
1	MLOps Engineer and Data Scientist (PagerDuty)	All alerts
2	Engineering manager	Both people unreachable for 30 min on a P1; cross-team coordination needed
3	Compliance, Legal, Azure Support	Suspected PAN exposure or GDPR breach; regulator request; Azure platform outage

7.6 Production change checklist

Every production change needs: a pull request reviewed by the other person with the rollback target named; a slot inside the deployment window and outside freezes (or an emergency override with an incident ID); a canary with armed abort rules for any model or scoring-code change; and, afterwards, a clean health report, no release-caused drift alerts, the 24 h stability tag, and an updated runbook if a procedure changed.

8. Package Self-Check

Assignment requirement	Where it is met
Assumptions and constraints page; missing information stated	Section 1 (C1–C10, A1–A13, open items)
Diagram: ingestion, feature store, training compute, registry, online and streaming targets, telemetry, storage tiers	Figure 1; deployment targets in Section 2.4; editable .drawio file
Components annotated with responsibilities, throughput targets, interfaces	Figure 1 labels; Tables 2-1 to 2-4 (Sections 2.4–2.7)
MLflow experiment naming, artifact paths, tagging, evaluation gating, sample config	Sections 3.1–3.5; Appendix A
GitHub Actions: test, build, validate, promote, deploy, rollback; YAML excerpts; gating conditions	Section 4; Figure 2; Table 4-1; Appendix B
Telemetry metrics and streaming aggregation architecture	Sections 5.1–5.2; Appendix E

Assignment requirement	Where it is met
PSI, KL, KS with numeric thresholds and reasons tied to business risk and volume	Sections 5.3–5.4; Appendix D
Alerting, automated responses, retraining triggers	Sections 5.5–5.6
Storage tiers, lifecycle policies, retention values, archival steps	Section 6; Appendix G
Runbook: daily checks, incident response, retraining, deployment for two people	Section 7; Appendix H
2–3 sentence design rationale	First page of the body; Dennis_DesignRationale.txt
Appendix configurations match the design assumptions	Appendix I; enforced in CI by `check_config_consistency.py`

Incident traceability

Incident	Controls in this design	Detection and recovery now
#1 Silent drift (6 days, $340K)	PFR ratio, unseen-category rate, hourly hour-matched drift tests, automatic challenger retrain, promotion hold	Critical within 1 h; retrain dispatched automatically
#2 Rollback failure (23 min, 4,200 transactions delayed)	G3 pre-traffic load test, canary abort every 60 s, warm standby, rollback controller	Caught before traffic; worst case under 5 min on 10% of traffic
#3 Storage spike (+340%, 4.2 TB redundant)	90-day experiment deletion, lifecycle on every container, version clean-up, budget alerts, inventory	Experiment store bounded near 1.2 TB; growth > 20% flagged daily
#4 Feature store latency (450 ms, 8% over SLA)	HA clustered Azure Managed Redis, 25 ms timeout with fallback, fallback-rate alert, seasonal capacity check	Lookup time capped at 25 ms; alert at 1% fallback

Appendix: Sample Configurations

Paths are repository paths in `fraudshield-mlops`. Every value matches the body of this package; Appendix I cross-references them. Listings marked excerpt omit only boilerplate.

A. MLflow configuration

A.1 config/mlflow_run_contract.yaml

```
# config/mlflow_run_contract.yaml
# Enforced by scripts/check_run_contract.py in ml-pipeline-validate (gate G7).
version: 2.0.0

tracking:
  # Resolved at runtime from MLClient(...).workspaces.get(ws).mlflow_tracking_uri (Appendix A.2);
  # inside Azure ML jobs the URI and credentials are injected, so nothing is hard-coded.
  workspace: fraud-ml-workspace
  resource_group: fraud-ml-rg
  region: eastus2

naming:
  experiment_pattern: "fraud-detection-{task}-v{major}"   # lowercase kebab-case
  tasks: [classification, anomaly, monitoring]
  experiment_current: fraud-detection-classification-v3    # FraudShield-v3, 47-feature set v3
  new_major_when: [feature set changes, model architecture changes, label definition changes]
  run_pattern: "{model_type}-{config_id}-{yyyymmdd}-{hhmmss}"   # UTC
  run_example: xgboost-tuned-optuna-20261011-020412
  model_types: [xgboost, lightgbm, randomforest, neuralnet]
  config_ids: [baseline, tuned-optuna, tuned-hyperopt, warmstart, ensemble]
  sweeps: "parent run = sweep, child runs = trials; trials are never registered"

required_params:            # 20
  - framework.xgboost_version      # 2.1.0
  - framework.sklearn_version      # 1.5.0
  - model.n_estimators
  - model.max_depth
  - model.learning_rate
  - model.subsample
  - model.colsample_bytree
  - model.reg_alpha
  - model.reg_lambda
  - model.early_stopping_rounds
  - model.decision_threshold       # 0.5 today; owned by the Data Scientist
  - data.version                   # Azure ML data asset version, e.g. fraud-training-data:42
  - data.feature_set               # v3.0 (47 features)
  - data.window_start
  - data.window_end                # D-3, so labels are >= 72 h mature
  - data.train_rows
  - data.test_rows
  - data.positive_ratio
  - seed.global                    # 42: numpy, random, xgboost random_state
  - env.image_digest               # sha256 of the scoring/training image in fraudmlacr

required_metrics:           # 16, computed on the 30-day holdout
  - eval.auc_roc                   # gate G1 >= 0.94
  - eval.precision_at_0.5          # gate G2 >= 0.82
  - eval.recall_at_0.5             # gate G2 >= 0.91
  - eval.f1_score                  # gate G2 >= 0.86
  - eval.precision_at_0.7
  - eval.recall_at_0.7
  - eval.false_positive_rate       # gate G4 <= 0.02
  - eval.false_negative_rate
  - eval.brier_score               # calibration; KL on scores assumes calibrated output
```

```
  - eval.expected_cost_per_1k      # gate G9: (127 x FN + 8 x FP) per 1,000 txns
  - eval.champion_expected_cost_per_1k   # champion scored on the same holdout
  - eval.model_latency_p50_ms      # offline single-row benchmark (informational)
  - eval.model_latency_p99_ms      # end-to-end latency is gate G3 (endpoint load test)
  - eval.shap_stability            # gate G5 >= 0.85
  - fairness.demographic_parity_diff   # gate G6 <= 0.05
  - fairness.equalized_odds_diff       # gate G6 <= 0.05

required_tags:              # 12
  project: fraud-detection
  model_type: xgboost
  pipeline_version: "<repo release semver>"
  git_commit: "<40-char sha>"
  run.trigger: [scheduled, drift, performance, manual, code-change]
  run.trigger_ref: "<alert id | commit sha | ticket id>"
  owner: "<Entra UPN of submitter, or ml-release>"
  data_source: event-hubs-capture
  data_region: eastus2              # westeurope for the EU stamp; EU data never leaves it
  compliance.pci_dss: "true"        # PAN never present; keyed card hash only
  compliance.soc2: "true"
  compliance.gdpr: "true"

required_artifacts:
  model/: "MLflow xgboost flavour + signature"
  preprocessing/: "fitted sklearn pipeline + preprocessing.sha256"
  shap/: "global_importance.csv, summary.png, attribution_baseline.json"
  drift-baseline/: "bin_edges.json (20 equal-frequency bins), profiles_48.parquet (hour x day type)"
  evaluation/: "confusion_matrix.png, roc_curve.png, precision_recall.png, calibration.png,
                segment_metrics.csv"
  environment/: "conda.yaml, requirements.lock (hashed), image_digest.txt"
  lineage/: "lineage.json: data asset+version, feature pipeline commit, training job id, sweep id"

registry:
  name: fraud-detection-xgboost
  version: "auto-increment integer assigned by Azure ML (never set by hand)"
  lifecycle_tags:             # MLflow stages are deprecated upstream; lifecycle is recorded as tags
    lifecycle: [staging, rejected, approved, production, standby, archived, rolled-back]
    role: [champion, challenger, previous]
    retention: [rejected, staging, production]   # mirrored as blob index tag on the package in models/
    semver: "3.MINOR.PATCH"   # MINOR = retrain on new data, PATCH = scoring code or threshold change
    other: [source_run_id, data.version, approved_by, approved_at, deployed_at, stability]

retention:                  # implemented by the lifecycle policy in Appendix G
  run_artifacts_days: 90    # experiments container; registered packages are copied out first
  model_packages_days: {rejected: 90, staging: 730, production: 730}
  production_audit_copy_days: 2557   # audit-7y, immutable
```

A.2 Connecting a notebook or local script to the workspace tracking server

```
import mlflow
from azure.ai.ml import MLClient
from azure.identity import DefaultAzureCredential

ml_client = MLClient(DefaultAzureCredential(), subscription_id="<subscription-id>",
                     resource_group_name="fraud-ml-rg", workspace_name="fraud-ml-workspace")
mlflow.set_tracking_uri(ml_client.workspaces.get(ml_client.workspace_name).mlflow_tracking_uri)
mlflow.set_experiment("fraud-detection-classification-v3")
```

A.3 src/fraudshield/train/log_run.py (logging pattern of the training step)

```
# src/fraudshield/train/log_run.py  (runs as the "train" step of pipelines/train-fraud-model.yml)
# Logs exactly the contract in config/mlflow_run_contract.yaml. It does NOT register the model:
# registration happens after the gates in ml-pipeline-validate, so gate logic lives in one place.
```

```
import os, random
from datetime import datetime, timezone

import mlflow, numpy as np, xgboost as xgb, sklearn
from sklearn import metrics as skm
from fraudshield.explain import shap_stability, write_shap_artifacts
from fraudshield.monitoring import write_drift_baseline
from fraudshield.fairness import segment_gaps
from fraudshield.lineage import write_lineage, sha256_of

SEED = 42
random.seed(SEED); np.random.seed(SEED)
COST_FN, COST_FP = 127.0, 8.0                      # scenario cost model, Section 8


def expected_cost_per_1k(y, yhat):
    fn = int(((y == 1) & (yhat == 0)).sum()); fp = int(((y == 0) & (yhat == 1)).sum())
    return 1000.0 * (COST_FN * fn + COST_FP * fp) / len(y)


def run(cfg, data, champion):
    X_tr, y_tr, X_te, y_te = data.train_test()
    mlflow.set_experiment("fraud-detection-classification-v3")
    run_name = f"xgboost-{cfg['config_id']}-{datetime.now(timezone.utc):%Y%m%d-%H%M%S}"
    with mlflow.start_run(run_name=run_name):
        mlflow.set_tags({
            "project": "fraud-detection", "model_type": "xgboost",
            "pipeline_version": os.environ["PIPELINE_VERSION"], "git_commit": os.environ["GIT_COMMIT"],
            "run.trigger": os.environ["RUN_TRIGGER"], "run.trigger_ref": os.environ["RUN_TRIGGER_REF"],
            "owner": os.environ.get("SUBMITTED_BY", "ml-release"), "data_source": "event-hubs-capture",
            "data_region": "eastus2", "compliance.pci_dss": "true", "compliance.soc2": "true",
            "compliance.gdpr": "true"})
        hp = cfg["hyperparameters"]          # n_estimators=200, max_depth=8, learning_rate=0.05, ...
        mlflow.log_params({
            "framework.xgboost_version": xgb.__version__, "framework.sklearn_version": sklearn.__version__,
            **{f"model.{k}": v for k, v in hp.items()},                    # 7 model.* params
            "model.early_stopping_rounds": 20, "model.decision_threshold": cfg["decision_threshold"],
            "data.version": data.asset_version, "data.feature_set": data.feature_set,
            "data.window_start": data.window_start, "data.window_end": data.window_end,
            "data.train_rows": len(X_tr), "data.test_rows": len(X_te),
            "data.positive_ratio": round(float(y_tr.mean()), 4),
            "seed.global": SEED, "env.image_digest": os.environ["IMAGE_DIGEST"]})

        model = xgb.XGBClassifier(**hp, random_state=SEED, eval_metric="aucpr", early_stopping_rounds=20)
        model.fit(X_tr, y_tr, eval_set=[(X_te, y_te)], verbose=False)
        p = model.predict_proba(X_te)[:, 1]
        yh = {t: (p >= t).astype(int) for t in (0.5, 0.7, cfg["decision_threshold"])}
        tn, fp, fn, tp = skm.confusion_matrix(y_te, yh[cfg["decision_threshold"]]).ravel()
        # segments approved by Compliance: country, channel, amount band
        gaps = segment_gaps(y_te, yh[cfg["decision_threshold"]], data.segments_te)
        mlflow.log_metrics({
            "eval.auc_roc": skm.roc_auc_score(y_te, p),
            "eval.precision_at_0.5": skm.precision_score(y_te, yh[0.5]),
            "eval.recall_at_0.5": skm.recall_score(y_te, yh[0.5]),
            "eval.f1_score": skm.f1_score(y_te, yh[cfg["decision_threshold"]]),
            "eval.precision_at_0.7": skm.precision_score(y_te, yh[0.7]),
            "eval.recall_at_0.7": skm.recall_score(y_te, yh[0.7]),
            "eval.false_positive_rate": fp / (fp + tn), "eval.false_negative_rate": fn / (fn + tp),
            "eval.brier_score": skm.brier_score_loss(y_te, p),
            "eval.expected_cost_per_1k": expected_cost_per_1k(y_te, yh[cfg["decision_threshold"]]),
            "eval.champion_expected_cost_per_1k": champion.expected_cost_per_1k(X_te, y_te),
            **data.latency_benchmark(model),            # eval.model_latency_p50_ms / _p99_ms
            "eval.shap_stability": shap_stability(model, X_te, n_boot=5, seed=SEED),
```

```
            "fairness.demographic_parity_diff": gaps["flag_rate_gap"],
            "fairness.equalized_odds_diff": gaps["equalized_odds_gap"]})

        mlflow.xgboost.log_model(model, artifact_path="model",
                                 signature=mlflow.models.infer_signature(X_te.head(5), p[:5]))
        mlflow.log_artifacts(data.preprocessing_dir, artifact_path="preprocessing")
        mlflow.log_text(sha256_of(data.preprocessing_dir), "preprocessing/preprocessing.sha256")
        write_shap_artifacts(model, X_te, out="shap")                          # uses pred_contribs=True
        write_drift_baseline(model, X_tr, p_train=model.predict_proba(X_tr)[:, 1], out="drift-baseline")
        mlflow.log_artifacts("shap", "shap"); mlflow.log_artifacts("drift-baseline", "drift-baseline")
        mlflow.log_artifacts(data.evaluation_dir(y_te, p), "evaluation")
        mlflow.log_artifacts("environment", "environment")
        write_lineage(out="lineage/lineage.json", data=data, job_id=os.environ["AZUREML_RUN_ID"])
        mlflow.log_artifacts("lineage", "lineage")
```

A.4 scripts/package_and_register.sh

```
#!/usr/bin/env bash
# scripts/package_and_register.sh RUN_ID STATE(staging|rejected) SEMVER DATA_VERSION
# Copies the run's artifacts out of the experiments container (deleted at 90 days) into the
# models container, sets the blob index tag that drives retention, then registers that copy.
set -euo pipefail
RUN_ID=$1; STATE=$2; SEMVER=$3; DATA_VERSION=$4
MODEL=fraud-detection-xgboost
SRC="https://fraudmlstore.blob.core.windows.net/experiments/ExperimentRun/dcid.${RUN_ID}/"
DST="https://fraudmlstore.blob.core.windows.net/models/${MODEL}/${SEMVER}/"

azcopy login --identity                                    # runner's managed identity
azcopy copy "${SRC}*" "${DST}" --recursive \
  --blob-tags="retention=${STATE}&model=${MODEL}&run=${RUN_ID}"

az ml model create --name "${MODEL}" --type mlflow_model \
  --path "azureml://datastores/models/paths/${MODEL}/${SEMVER}/model" \
  --tags lifecycle="${STATE}" retention="${STATE}" semver="${SEMVER}" \
         source_run_id="${RUN_ID}" data.version="${DATA_VERSION}" \
  --query version -o tsv > version.txt                     # version is auto-incremented

# Champion lookup used by validate/deploy (tags, not MLflow stages):
#   az ml model list -n fraud-detection-xgboost --query "[?tags.role=='champion'].version | [0]" -o tsv
# On full promotion (finalize step in ml-pipeline-deploy):
#   az ml model update -n $MODEL -v $NEW  --set tags.lifecycle=production tags.role=champion \
#                                             tags.deployed_at=$(date -u +%FT%TZ)
#   az ml model update -n $MODEL -v $PREV --set tags.lifecycle=standby tags.role=previous
#   azcopy set-properties "${DST}*" --recursive \
#          --blob-tags="retention=production&model=${MODEL}&run=${RUN_ID}"
```

B. GitHub Actions workflows

B.1 .github/workflows/ml-release.yml

```
# .github/workflows/ml-release.yml  - orchestrator
# One run carries a candidate from test to production. The stages are reusable workflows called
# with `needs:`, because chained `workflow_run` triggers stop after three levels and would never
# reach Deploy, and because cross-workflow artifacts need run IDs that `needs` passes for free.
name: ml-release

on:
  schedule:
    - cron: "0 2 * * 0"                 # weekly retrain: Sunday 02:00 UTC (A5)
  repository_dispatch:
    types: [drift-retrain]              # sent by func-fraud-ops on Medium+ drift or performance drop
  workflow_dispatch:
    inputs:
```

```
      reason:
        description: "Why this release (ticket or alert ID)"
        required: true
        type: string
      release_type:
        type: choice
        options: [retrain, code-only]    # code-only = PATCH: same model, new scoring image
        default: retrain
      emergency:
        description: "Bypass the change-freeze calendar (incident ID required in reason)"
        type: boolean
        default: false

permissions:
  id-token: write                        # OIDC federation to Azure; no stored service-principal secret
  contents: read
  actions: read

concurrency:
  group: release-fraud-scoring-prod      # one release at a time; rollback uses its own group
  cancel-in-progress: false

jobs:
  test:
    uses: ./.github/workflows/ml-pipeline-test.yml

  build:
    needs: test
    uses: ./.github/workflows/ml-pipeline-build.yml
    with:
      trigger: >-
        ${{ github.event_name == 'repository_dispatch' && 'drift'
        || github.event_name == 'schedule' && 'scheduled' || 'manual' }}
      trigger_ref: ${{ github.event.client_payload.alert_id || inputs.reason || github.sha }}
      release_type: ${{ inputs.release_type || 'retrain' }}
    secrets: inherit

  validate:
    needs: build
    uses: ./.github/workflows/ml-pipeline-validate.yml
    with:
      run_id: ${{ needs.build.outputs.run_id }}
      data_version: ${{ needs.build.outputs.data_version }}
    secrets: inherit

  promote:
    needs: [build, validate]
    uses: ./.github/workflows/ml-pipeline-promote.yml
    with:
      model_version: ${{ needs.validate.outputs.model_version }}
      data_window_end: ${{ needs.build.outputs.data_window_end }}
    secrets: inherit

  deploy:
    needs: [validate, promote]
    uses: ./.github/workflows/ml-pipeline-deploy.yml
    with:
      model_version: ${{ needs.validate.outputs.model_version }}
      emergency: ${{ inputs.emergency || false }}
    secrets: inherit
```

B.2 .github/workflows/ml-pipeline-test.yml

```
# .github/workflows/ml-pipeline-test.yml  (PR gate + first stage of ml-release)
name: ml-pipeline-test
on:
  pull_request:
    branches: [main]
  push:
    branches: [develop]
  workflow_call:

permissions:
  contents: read

jobs:
  lint:
    runs-on: ubuntu-latest               # no Azure access needed: GitHub-hosted is fine
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install black==24.4.2 isort==5.13.2 flake8==7.0.0 yamllint==1.35.1
      - run: black --check src tests && isort --check-only src tests
      - run: flake8 src tests --max-line-length=100 --extend-ignore=E203,W503
      - run: yamllint -d relaxed config/ .github/workflows/

  unit-and-contract:
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements-dev.txt && pip install -e .
      - name: Unit tests, coverage gate 85 %
        run: pytest tests/unit --cov=src --cov-report=term-missing --cov-fail-under=85
      - name: Schema contracts (training, inference, feature store)
        run: python scripts/validate_schema.py --all data/schemas/
      - name: PAN guard - PAN-like fields or Luhn-valid 13-19 digit values must be rejected
        run: pytest tests/contract/test_pan_guard.py
      - name: Offline/online feature parity on 1,000 golden records
        run: pytest tests/contract/test_feature_parity.py --tolerance 1e-6
      - name: Gates, drift thresholds and run contract agree (names, units, counts)
        run: python scripts/check_config_consistency.py config/

  iac:
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform -chdir=infra fmt -check -recursive
      - run: terraform -chdir=infra init -backend=false && terraform -chdir=infra validate
```

B.3 .github/workflows/ml-pipeline-build.yml (excerpt)

```
# .github/workflows/ml-pipeline-build.yml  (excerpt)
name: ml-pipeline-build
on:
  workflow_call:
    inputs:
      trigger:        {type: string, required: true}
```

```
      trigger_ref:    {type: string, required: true}
      release_type:   {type: string, default: retrain}
    outputs:
      run_id:
        value: ${{ jobs.train.outputs.run_id }}
      data_version:
        value: ${{ jobs.train.outputs.data_version }}
      data_window_end:
        value: ${{ jobs.train.outputs.window_end }}

permissions:
  id-token: write
  contents: read

jobs:
  image:
    runs-on: [self-hosted, linux, fraud-ml-vnet]   # private endpoints: ACR, workspace, storage
    outputs:
      digest: ${{ steps.push.outputs.digest }}
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
      - id: push
        run: |
          az acr login -n fraudmlacr
          IMG=fraudmlacr.azurecr.io/fraudshield-scoring:${GITHUB_SHA}
          docker build -f docker/Dockerfile -t "$IMG" . && docker push "$IMG"
          DIGEST=$(docker inspect --format='{{index .RepoDigests 0}}' "$IMG" | cut -d@ -f2)
          echo "digest=$DIGEST" >> "$GITHUB_OUTPUT"

  train:
    needs: image
    if: inputs.release_type == 'retrain'
    runs-on: [self-hosted, linux, fraud-ml-vnet]
    timeout-minutes: 180
    outputs:
      run_id: ${{ steps.submit.outputs.run_id }}
      data_version: ${{ steps.submit.outputs.data_version }}
      window_end: ${{ steps.window.outputs.end }}
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
      - run: |
          az extension add -n ml -y
          az configure --defaults group=fraud-ml-rg workspace=fraud-ml-workspace
      - id: window
        name: Training window = 90 days ending D-3 (labels at least 72 h mature)
        run: echo "end=$(date -u -d '3 days ago' +%F)" >> "$GITHUB_OUTPUT"
      - id: submit
        name: Submit Azure ML pipeline (snapshot -> train -> evaluate -> SHAP -> drift baseline)
        run: |
          JOB=$(az ml job create -f pipelines/train-fraud-model.yml \
                 --set inputs.window_end=${{ steps.window.outputs.end }} inputs.window_days=90 \
                       inputs.image_digest=${{ needs.image.outputs.digest }} \
                       tags.run.trigger=${{ inputs.trigger }} tags.run.trigger_ref=${{ inputs.trigger_ref }} \
```

```
                      tags.git_commit=${GITHUB_SHA} \
                --query name -o tsv)
          timeout 9000 az ml job stream -n "$JOB" > /dev/null || true
          STATUS=$(az ml job show -n "$JOB" --query status -o tsv)
          [ "$STATUS" = "Completed" ] || { echo "::error::training pipeline $JOB ended $STATUS"; exit 1; }
          RUN_ID=$(az ml job list --parent-job-name "$JOB" \
                    --query "[?display_name=='train'] | [0].name" -o tsv)
          echo "run_id=$RUN_ID" >> "$GITHUB_OUTPUT"
          DATA=$(az ml job show -n "$RUN_ID" --query 'inputs.training_data.path' -o tsv)
          echo "data_version=$DATA" >> "$GITHUB_OUTPUT"
```

B.4 .github/workflows/ml-pipeline-validate.yml (excerpt)

```
# .github/workflows/ml-pipeline-validate.yml  (excerpt)
name: ml-pipeline-validate
on:
  workflow_call:
    inputs:
      run_id:       {type: string, required: true}
      data_version: {type: string, required: true}
    outputs:
      model_version:
        value: ${{ jobs.gates.outputs.model_version }}

permissions:
  id-token: write
  contents: read

jobs:
  gates:
    runs-on: [self-hosted, linux, fraud-ml-vnet]
    outputs:
      model_version: ${{ steps.register.outputs.model_version }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements-ci.txt
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}

      - name: G7 - run contract (20 params, 16 metrics, 12 tags, artifacts, lineage resolvable)
        run: |
          python scripts/check_run_contract.py --run-id ${{ inputs.run_id }} \
                 --contract config/mlflow_run_contract.yaml

      - name: G1 G2 G4 G5 G6 G9 - offline gates from config/gates.yaml (champion scored on the same holdout)
        id: gates
        run: |
          python scripts/validate_gates.py --run-id ${{ inputs.run_id }} --gates config/gates.yaml \
                 --stage offline --champion "$(az ml model list -n fraud-detection-xgboost \
                 --query "[?tags.role=='champion'].version | [0]" -o tsv)" --report validation_report.md

      - name: Package and register (passed -> staging, failed -> rejected, kept 90 days for debugging)
        id: register
        if: always() && steps.gates.outcome != 'skipped'
        run: |
          STATE=$([ "${{ steps.gates.outcome }}" = "success" ] && echo staging || echo rejected)
          SEMVER=$(python scripts/next_semver.py --kind minor)
```

```
          bash scripts/package_and_register.sh "${{ inputs.run_id }}" "$STATE" "$SEMVER" \
               "${{ inputs.data_version }}"
          echo "model_version=$(cat version.txt)" >> "$GITHUB_OUTPUT"

      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: validation-report
          path: validation_report.md

      - name: Notify rejection
        if: failure()
        run: |
          python scripts/notify.py --channel ml-alerts --severity medium \
                 --title "Candidate rejected" --file validation_report.md
```

B.5 .github/workflows/ml-pipeline-promote.yml

```
# .github/workflows/ml-pipeline-promote.yml
name: ml-pipeline-promote
on:
  workflow_call:
    inputs:
      model_version:   {type: string, required: true}
      data_window_end: {type: string, required: true}

permissions:
  id-token: write
  contents: read
  actions: read

jobs:
  hold-check:
    runs-on: ubuntu-latest
    steps:
      - name: Drift promotion hold (repo variable set by func-fraud-ops on High/Critical drift)
        env:
          HOLD: ${{ vars.PROMOTION_HOLD_SINCE }}          # ISO date of drift onset, empty when clear
          WINDOW_END: ${{ inputs.data_window_end }}
        run: |
          if [ -n "$HOLD" ] && [[ "$WINDOW_END" < "${HOLD:0:10}" ]]; then
            echo "::error::Promotion hold since $HOLD; candidate data ends $WINDOW_END (pre-onset)."
            exit 1
          fi

  approve:                                  # gate G8
    needs: hold-check
    runs-on: ubuntu-latest
    environment:
      name: production                      # reviewers: Data Scientist, MLOps Engineer (1 of 2);
                                            # "prevent self-review" on; branch main only
    steps:
      - run: echo "Approved for canary release of model version ${{ inputs.model_version }}"

  record:
    needs: approve
    runs-on: [self-hosted, linux, fraud-ml-vnet]
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
```

```
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
      - name: Capture the approver from GitHub and write the approval record
        env:
          GH_TOKEN: ${{ github.token }}
        run: |
          az extension add -n ml -y
          APPROVER=$(gh api repos/${{ github.repository }}/actions/runs/${{ github.run_id }}/approvals \
                    --jq '[.[] | select(.state=="approved")][0].user.login')
          az ml model update -n fraud-detection-xgboost -v ${{ inputs.model_version }} \
             -g fraud-ml-rg -w fraud-ml-workspace \
             --set tags.lifecycle=approved tags.approved_by=$APPROVER \
                   tags.approved_at=$(date -u +%FT%TZ) tags.release_run=${{ github.run_id }}
          python scripts/write_audit_record.py --type approval --model-version ${{ inputs.model_version }} \
             --actor "$APPROVER" --run-id ${{ github.run_id }}     # -> fraudmlaudit/audit-7y/approvals/
```

B.6 .github/workflows/ml-pipeline-deploy.yml (key steps)

```
# .github/workflows/ml-pipeline-deploy.yml  (key steps)
# Slots alternate: the candidate goes into whichever of blue/green is NOT live, replacing the old standby.
name: ml-pipeline-deploy
on:
  workflow_call:
    inputs:
      model_version: {type: string, required: true}
      emergency:     {type: boolean, default: false}

permissions:
  id-token: write
  contents: read

env:
  EP: fraud-scoring-prod
  MODEL: fraud-detection-xgboost

jobs:
  canary-release:
    if: github.ref == 'refs/heads/main'
    runs-on: [self-hosted, linux, fraud-ml-vnet]
    timeout-minutes: 300
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
      - run: |
          az extension add -n ml -y
          az configure --defaults group=fraud-ml-rg workspace=fraud-ml-workspace
          pip install -r requirements-ci.txt

      - name: Deployment window and change freeze (Tue-Thu 13:00-17:00 UTC; Black Friday, month-end freezes)
        run: |
          python scripts/change_calendar.py --config config/change_calendar.yaml \
                 --emergency ${{ inputs.emergency }}

      - name: Resolve slots
        id: slots
        run: |
          read LIVE TARGET < <(az ml online-endpoint show -n $EP --query traffic -o json \
            | python3 -c 'import json,sys; t=json.load(sys.stdin); l=max(t,key=t.get)
          print(l, "green" if l=="blue" else "blue")')
          echo "live=$LIVE" >> "$GITHUB_OUTPUT"; echo "target=$TARGET" >> "$GITHUB_OUTPUT"
```

```
          # during the release the rollback target is the current live slot
          az ml online-endpoint update -n $EP --set tags.rollback_target=$LIVE tags.release_state=deploying

      - name: Create candidate in the target slot at 0 % traffic (replaces the old standby)
        run: |
          DIGEST=$(az ml model show -n $MODEL -v ${{ inputs.model_version }} \
                   --query 'tags."env.image_digest"' -o tsv)
          az ml online-deployment delete -n ${{ steps.slots.outputs.target }} -e $EP --yes 2>/dev/null || true
          az ml online-deployment create -f deploy/candidate-deployment.yml -e $EP \
             --name ${{ steps.slots.outputs.target }} \
             --set model=azureml:$MODEL:${{ inputs.model_version }} \
                   environment.image=fraudmlacr.azurecr.io/fraudshield-scoring@$DIGEST

      - name: G3 - pre-traffic load test at 28 tps per instance for 10 min (synthetic, header-routed)
        run: |
          python scripts/load_test.py --endpoint $EP --deployment ${{ steps.slots.outputs.target }} \
                 --gates config/gates.yaml --gate G3 --replay audit-sample-24h --synthetic-header

      - name: G10 - shadow - mirror 20 % of live traffic for 30 min, compare with live
        run: |
          az ml online-endpoint update -n $EP --mirror-traffic "${{ steps.slots.outputs.target }}=20" \
                                              --set tags.release_state=shadow
          python scripts/release_controller.py observe --stage shadow --gates config/gates.yaml \
                 --live ${{ steps.slots.outputs.live }} --candidate ${{ steps.slots.outputs.target }}
          az ml online-endpoint update -n $EP --mirror-traffic "${{ steps.slots.outputs.target }}=0"

      - name: G11 - canary - 10 % live traffic until >= 5,000 requests (max 180 min), abort rules every 60 s
        run: |
          az ml online-endpoint update -n $EP \
             --traffic "${{ steps.slots.outputs.live }}=90 ${{ steps.slots.outputs.target }}=10" \
             --set tags.release_state=canary
          # On an abort rule the controller itself restores live=100 within one poll, then exits 1.
          python scripts/release_controller.py observe --stage canary --gates config/gates.yaml \
                 --live ${{ steps.slots.outputs.live }} --candidate ${{ steps.slots.outputs.target }}

      - name: Full promotion - scale, move autoscale, switch traffic, keep previous as warm standby
        run: |
          L=${{ steps.slots.outputs.live }}; T=${{ steps.slots.outputs.target }}
          az ml online-deployment update -n $T -e $EP --set instance_count=3
          bash scripts/autoscale.sh $T                        # min 3, max 6, Black Friday profile
          az ml online-endpoint update -n $EP --traffic "$L=0 $T=100" \
             --set tags.rollback_target=$L tags.release_state=post-release tags.release_at=$(date -u +%FT%TZ)
          az ml online-deployment update -n $L -e $EP --set instance_count=2      # standby, kept 7 days

      - name: Registry tags, lineage manifest and audit copy
        run: python scripts/finalize_release.py --model-version ${{ inputs.model_version }} \
                    --standby-slot ${{ steps.slots.outputs.live }} --audit-container audit-7y

      - name: Restore live slot on any failure
        if: failure()
        run: |
          az ml online-endpoint update -n $EP --mirror-traffic "${{ steps.slots.outputs.target }}=0" || true
          az ml online-endpoint update -n $EP \
             --traffic "${{ steps.slots.outputs.live }}=100 ${{ steps.slots.outputs.target }}=0" \
             --set tags.release_state=aborted
          python scripts/notify.py --channel ml-alerts --severity high \
                 --title "Release aborted, live slot restored"
```

B.7 .github/workflows/ml-pipeline-rollback.yml

```
# .github/workflows/ml-pipeline-rollback.yml
# Two entry points, one procedure:
#   - repository_dispatch from func-fraud-ops AFTER it has already switched traffic (auto path, < 5 min)
```

```
#   - workflow_dispatch by either team member (manual path, same speed because standby is warm)
name: ml-pipeline-rollback
on:
  workflow_dispatch:
    inputs:
      reason:
        description: "Reason and incident/alert ID (written to the audit record)"
        required: true
        type: string
  repository_dispatch:
    types: [auto-rollback-triggered]

permissions:
  id-token: write
  contents: read

concurrency:
  group: rollback-fraud-scoring-prod
  cancel-in-progress: false

jobs:
  rollback:
    runs-on: [self-hosted, linux, fraud-ml-vnet]
    environment: production-rollback     # no reviewers: pre-approved by policy RB-1; main and hotfix/* only
    timeout-minutes: 30
    env:
      EP: fraud-scoring-prod
      REASON: >-
        ${{ inputs.reason || format('auto - alert {0}, rule {1}',
        github.event.client_payload.alert_id, github.event.client_payload.rule) }}
      ACTOR: >-
        ${{ github.event_name == 'repository_dispatch' && 'func-fraud-ops (policy RB-1)' || github.actor }}
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
      - run: |
          az extension add -n ml -y
          az configure --defaults group=fraud-ml-rg workspace=fraud-ml-workspace

      - name: Switch all traffic to the rollback target (idempotent, about 1-2 min)
        id: switch
        run: |
          TARGET=$(az ml online-endpoint show -n $EP --query tags.rollback_target -o tsv)
          FROM=$(az ml online-endpoint show -n $EP --query traffic -o json | python3 -c \
                 'import json,sys; t=json.load(sys.stdin); print(max(t,key=t.get))')
          echo "from=$FROM" >> "$GITHUB_OUTPUT"; echo "to=$TARGET" >> "$GITHUB_OUTPUT"
          if [ "$FROM" = "$TARGET" ]; then
            echo "Traffic already on $TARGET (auto path completed the switch)"
          elif az ml online-deployment show -n "$TARGET" -e $EP > /dev/null 2>&1; then
            az ml online-endpoint update -n $EP --mirror-traffic "$FROM=0" --traffic "$TARGET=100 $FROM=0"
          else
            echo "::warning::No warm standby; slow path: redeploying previous champion (about 15 min)"
            python scripts/release_controller.py redeploy-previous --endpoint $EP
          fi
          az ml online-endpoint update -n $EP --set tags.release_state=rolled-back

      - name: Restore capacity on the new live slot
        run: |
          az ml online-deployment update -n ${{ steps.switch.outputs.to }} -e $EP --set instance_count=3
```

```
          bash scripts/autoscale.sh ${{ steps.switch.outputs.to }}

      - name: Verify (synthetic smoke request + p99 of the last 5 min)
        run: python scripts/smoke_test.py --endpoint $EP --max-p99-ms 120 --synthetic-header

      - name: Audit record, registry tags, notifications
        env:
          PD_ROUTING_KEY: ${{ secrets.PAGERDUTY_ROUTING_KEY }}
        run: |
          python scripts/write_audit_record.py --type rollback --actor "$ACTOR" --reason "$REASON" \
                 --from-slot ${{ steps.switch.outputs.from }} --to-slot ${{ steps.switch.outputs.to }} \
                 --run-id ${{ github.run_id }}                    # -> audit-7y/rollbacks/
          python scripts/tag_rolled_back.py --slot ${{ steps.switch.outputs.from }}   # lifecycle=rolled-back
          python scripts/notify.py --channel incidents --severity critical \
                 --title "Rollback executed" --body "$REASON"
          jq -n --arg k "$PD_ROUTING_KEY" --arg s "FraudShield rollback: $REASON" \
             '{routing_key:$k, event_action:"trigger",
               payload:{summary:$s, source:"fraud-scoring-prod", severity:"critical"}}' \
          | curl -sf -X POST https://events.pagerduty.com/v2/enqueue \
                 -H 'Content-Type: application/json' -d @-
```

B.8 .github/workflows/audit-export.yml

```
# .github/workflows/audit-export.yml
# GitHub keeps Actions logs for at most 400 days; SOC 2 evidence here must be kept 2 years.
name: audit-export
on:
  schedule:
    - cron: "15 3 * * *"               # daily 03:15 UTC
  workflow_dispatch:

permissions:
  id-token: write
  contents: read
  actions: read

jobs:
  export:
    runs-on: [self-hosted, linux, fraud-ml-vnet]
    steps:
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
      - name: Export yesterday's completed runs, logs and approval history
        env:
          GH_TOKEN: ${{ github.token }}
        run: |
          DAY=$(date -u -d yesterday +%F); OUT=export/$(date -u -d yesterday +%Y/%m/%d); mkdir -p $OUT
          IDS=$(gh run list -R ${{ github.repository }} --created "$DAY" --status completed -L 500 \
                --json databaseId -q '.[].databaseId')
          for ID in $IDS; do
            gh api repos/${{ github.repository }}/actions/runs/$ID           > $OUT/$ID.run.json
            gh api repos/${{ github.repository }}/actions/runs/$ID/approvals  > $OUT/$ID.approvals.json
            gh api repos/${{ github.repository }}/actions/runs/$ID/logs       > $OUT/$ID.logs.zip
          done
          az storage blob upload-batch --auth-mode login --account-name fraudmlaudit \
             -d audit-2y --destination-path cicd -s export --overwrite false   # Cool after 1 day (lifecycle)
```

B.9 Environments, variables and secrets

Item	Setting

C. Promotion gates

C.1 config/gates.yaml

```
# config/gates.yaml  - single source of truth for every promotion gate (read by validate,
# load_test, release_controller and func-fraud-ops). Changes need DS + MLOps review (CODEOWNERS).
version: 1.2.0
model: fraud-detection-xgboost

cost_model:                      # scenario Section 8
  false_negative_usd: 127
  false_positive_usd: 8

offline:                         # ml-pipeline-validate, 30-day holdout, labels >= 72 h mature
  G1_auc_roc:             {metric: eval.auc_roc, op: ">=", value: 0.94}
  G2_f1:                  {metric: eval.f1_score, op: ">=", value: 0.86}
  G2_precision:           {metric: eval.precision_at_0.5, op: ">=", value: 0.82}
  G2_recall:              {metric: eval.recall_at_0.5, op: ">=", value: 0.91}
  G4_false_positive_rate: {metric: eval.false_positive_rate, op: "<=", value: 0.02}
  G5_shap_stability:      {metric: eval.shap_stability, op: ">=", value: 0.85}
  G6_demographic_parity:  {metric: fairness.demographic_parity_diff, op: "<=", value: 0.05,
                           max_worse_than_champion: 0.01}
  G6_equalized_odds:      {metric: fairness.equalized_odds_diff, op: "<=", value: 0.05,
                           max_worse_than_champion: 0.01}
  G9_expected_cost:       {metric: eval.expected_cost_per_1k, op: "<=",
                           ref: eval.champion_expected_cost_per_1k}

pre_traffic:                     # candidate slot at 0 % traffic, requests routed by deployment header
  G3_latency:
    rps_per_instance: 28         # scenario throughput floor per DS3_v2 instance
    instances: 2
    duration_s: 600
    p99_ms_max: 120              # constraint C2, measured end to end incl. Redis lookup and SHAP
    error_rate_max: 0.0

shadow:
  G10:
    mirror_percent: 20
    duration_min: 30
    pfr_ratio: [0.80, 1.25]      # candidate vs live predicted-fraud rate on the same requests
    score_psi_max: 0.10
    errors_max: 0

canary:
  G11:
    traffic_percent: 10
    min_requests: 5000           # ~70 min at average traffic; 0.1 % = at most 5 errors
    max_minutes: 180
    p99_ms_max: 120
    error_rate_max: 0.001
    pfr_ratio: [0.80, 1.25]

abort:                           # polled every 60 s in shadow/canary; alert rules use the same values
  latency:  {p99_ms_gt: 300, window_min: 3}
  errors:   {rate_gt: 0.005, min_failed: 5, window_min: 3}
  pfr_ratio_outside: [0.50, 2.00]

post_release:
  G12_stability_hours: 24        # auto-rollback stays armed for 24 h after full promotion
  standby: {instances: 2, keep_days: 7}
```

D. Drift thresholds and change calendar

D.1 config/drift_thresholds.yaml

```
# config/drift_thresholds.yaml  - read by func-fraud-ops drift-evaluator and perf-monitor
version: 1.0.0

baseline:
  source: "drift-baseline/ artifact of the serving model version (its own training snapshot)"
  profiles: "48 = hour_of_day x day_type (weekday | weekend)"
  numeric_bins: "20 equal-frequency bins; merged pairwise into 10 deciles for PSI and KL"
  smoothing_epsilon: 0.0001      # empty bins
  refresh: "with every model release (weekly), never on its own"

windows:
  distribution: {size: 1h, type: tumbling, persistence: 2}   # Medium/High need 2 consecutive windows
  operational:  {size: 5min, type: tumbling}
  labeled:      {schedule: "daily 06:00 UTC", cohort: "D-3", trend: "7-day rolling"}

features:
  numeric:      [amount_log, avg_txn_amount_30d, txn_count_24h, unique_merchants_7d,
                 time_since_last_txn_minutes, amount_vs_avg_ratio, velocity_score_1h, device_risk_score]
  categorical:  [merchantCategory_encoded, distance_flag, isOnline, geo_anomaly_flag]
  excluded:     [hour_of_day, day_of_week, is_weekend]   # constant inside an hour-matched window

thresholds:                        # [low, medium, high]
  psi:                   [0.10, 0.20, 0.25]       # all 12 monitored features + score
  ks_d:                  [0.10, 0.15, 0.20]       # 8 numeric features
  ks_alpha:              0.00625                  # 0.05 / 8, Bonferroni; D_crit = 0.016 at 12,600 txn/h
  kl_score:              [0.05, 0.10, 0.125]      # prediction-score distribution, 20 bins
  kl_attribution:        [0.05, 0.10, 0.125]      # share of mean |SHAP| across 47 features
  pfr_ratio_outside:     [[0.85, 1.18], [0.75, 1.33], [0.70, 1.43]]
  unseen_category_rate:  [0.005, 0.01, 0.03]
  feature_fallback_rate: [0.005, 0.01, 0.05]      # 5-min window; Redis timeouts
  labeled_recall_drop:   [0.01, 0.02, 0.03]
  labeled_auc_drop:      [0.01, 0.02, 0.03]
  labeled_f1_drop:       [0.02, 0.04, 0.06]

critical:                          # any one; fires on the first window
  - "pfr_ratio outside [0.50, 2.00]"
  - "unseen_category_rate >= 0.05"
  - "count(features at High PSI) >= 3 and kl_score >= 0.125"
  - "labeled_recall_drop >= 0.05"

actions:
  low:      [log, daily_digest]
  medium:   [slack, email, jira, challenger_retrain]
  high:     [pagerduty_primary, promotion_hold, challenger_retrain]
  critical: [pagerduty_both, promotion_hold, challenger_retrain,
             rollback_if_within_24h_of_release, notify_fraud_ops]

guardrails:
  max_drift_retrains_per_24h: 1
  seasonal_calendar: config/change_calendar.yaml   # distribution-only signals drop one level, no auto-retrain
  never_rollback_on_input_drift: true              # inputs are identical for live and candidate slots
```

D.2 config/change_calendar.yaml

```
# config/change_calendar.yaml  - read by ml-pipeline-deploy and by the drift-evaluator
# deploy window: both people online, between the 09-12 and 18-22 UTC peaks
deploy_window: {days: [Tue, Wed, Thu], start_utc: "13:00", end_utc: "17:00"}

freezes:
  - {name: black-friday-cyber-monday-2026, start: 2026-11-23, end: 2026-12-02}
```

```
  - {name: year-end, start: 2026-12-21, end: 2027-01-04}
  - {name: month-end, rule: "last 2 days and first day of every month"}

seasonal_events:                  # expected volume vs baseline peak (scenario Section 2)
  - {name: black-friday-cyber-monday-2026, start: 2026-11-26, end: 2026-12-01, volume_x: 3.5}
  - {name: month-end-salary, rule: "last 2 days and first day of every month", volume_x: 1.8}
  - {name: holiday-period, start: 2026-12-24, end: 2027-01-01, volume_x: 0.6}

emergency_override: "allowed only with an incident ID in the release reason; recorded in audit-2y"
```

E. Stream Analytics query

E.1 stream-analytics/fraud-telemetry-job.asaql

```
-- fraud-telemetry-job (Azure Stream Analytics), 3 SU baseline, 6 SU during seasonal events
-- Input   telemetry      : Event Hub fraud-events-ns/scoring-telemetry, consumer group "asa", JSON
-- Outputs auditpredictions : Blob fraudmlaudit/audit-7y, path predictions/{date}/{time}, Parquet
--         driftbins        : Azure Function func-fraud-ops/drift-ingest (batched HTTP)
--         opsagg           : Azure Function func-fraud-ops/ops-ingest
--         reviewqueue      : Service Bus queue fraud-review
-- Bin indices are computed by score.py against the serving model's own baseline, so the job
-- needs no reference data and the baseline always matches the model version being scored.

WITH Scored AS (
    SELECT * FROM telemetry TIMESTAMP BY event_ts
    WHERE is_synthetic = 0                      -- load-test and smoke requests never reach audit or drift
)

-- 1) Audit: every scored transaction (features, score, decision, SHAP, model version) for 7 years
SELECT request_id, event_ts, card_hash, deployment, model_version, is_mirror,
       score, decision, decision_threshold, features, shap_values, reason_codes,
       fallback_flags, latency_ms
INTO auditpredictions
FROM Scored

-- 2) Drift: hourly bin counts per monitored feature and for the score (live traffic only)
SELECT 'bins' AS record_type, System.Timestamp() AS window_end, s.model_version, s.deployment,
       s.hour_of_day, s.day_type, b.ArrayValue.f AS feature, b.ArrayValue.b AS bin, COUNT(*) AS n
INTO driftbins
FROM Scored s
CROSS APPLY GetArrayElements(s.bins) AS b
WHERE s.is_mirror = 0
GROUP BY TumblingWindow(hour, 1), s.model_version, s.deployment, s.hour_of_day, s.day_type,
         b.ArrayValue.f, b.ArrayValue.b

-- 3) Drift: hourly attribution mass per feature (KL on explanations)
SELECT 'attr' AS record_type, System.Timestamp() AS window_end, s.model_version, s.deployment,
       a.ArrayIndex AS feature_idx, SUM(ABS(a.ArrayValue)) AS abs_shap_sum, COUNT(*) AS n
INTO driftbins
FROM Scored s
CROSS APPLY GetArrayElements(s.shap_values) AS a
WHERE s.is_mirror = 0
GROUP BY TumblingWindow(hour, 1), s.model_version, s.deployment, a.ArrayIndex

-- 4) Operations: 5-minute aggregates per slot, live and mirrored kept apart (canary comparison)
SELECT System.Timestamp() AS window_end, deployment, model_version, is_mirror,
       COUNT(*) AS n,
       SUM(CAST(decision AS bigint)) AS flags,
       AVG(CAST(decision AS float)) AS pfr,
       AVG(CAST(fallback_flags.any AS float)) AS fallback_rate,
       AVG(CAST(category_unseen AS float)) AS unseen_category_rate,
       PERCENTILE_CONT(0.99) OVER (ORDER BY latency_ms) AS p99_server_ms
INTO opsagg
```

```
FROM Scored
GROUP BY TumblingWindow(minute, 5), deployment, model_version, is_mirror

-- 5) Business hand-off: high-risk transactions to fraud operations
SELECT request_id, event_ts, card_hash, score, reason_codes, model_version
INTO reviewqueue
FROM Scored
WHERE score >= 0.85 AND is_mirror = 0
```

F. Deployment and autoscale

F.1 deploy/candidate-deployment.yml

```
# deploy/candidate-deployment.yml  (managed online deployment; name/model/image set by the workflow)
$schema: https://azuremlschemas.azureedge.net/latest/managedOnlineDeployment.schema.json
name: green                                  # overridden with --name (slots alternate blue/green)
endpoint_name: fraud-scoring-prod
model: azureml:fraud-detection-xgboost:1     # overridden with the approved version
code_configuration:
  code: ../src/fraudshield/scoring
  scoring_script: score.py    # PAN guard > Redis > preprocess > XGBoost > TreeSHAP > telemetry
environment:
  image: fraudmlacr.azurecr.io/fraudshield-scoring@sha256:0000   # digest-pinned, overridden
instance_type: Standard_DS3_v2
instance_count: 2                            # candidate size; 3 + autoscale after promotion
environment_variables:
  REDIS_HOST: fraud-features.eastus2.redis.azure.net
  REDIS_TIMEOUT_MS: "25"                     # then fallback values + fallback flag
  TELEMETRY_EVENTHUB: fraud-events-ns.servicebus.windows.net/scoring-telemetry
  TELEMETRY_BATCH_MAX_MS: "200"              # background sender, never blocks the response
  REASON_CODES_TOP_K: "3"
request_settings:
  request_timeout_ms: 1000                   # gateway gives up at 250 ms anyway
  max_concurrent_requests_per_instance: 10    # Little's law: about 2 in flight per instance at 97 tps
liveness_probe:
  initial_delay: 30
  period: 10
  failure_threshold: 3
readiness_probe:
  initial_delay: 10
  period: 10
  success_threshold: 1
```

F.2 scripts/autoscale.sh

```
#!/usr/bin/env bash
# scripts/autoscale.sh SLOT  - attach the autoscale setting to whichever slot is live
set -euo pipefail
SLOT=$1; RG=fraud-ml-rg; AS=fraud-scoring-autoscale
ID=$(az ml online-deployment show -e fraud-scoring-prod -n "$SLOT" -g $RG -w fraud-ml-workspace \
     --query id -o tsv)

az monitor autoscale delete -g $RG -n $AS 2>/dev/null || true          # a setting cannot change target
az monitor autoscale create -g $RG -n $AS --resource "$ID" --min-count 3 --max-count 6 --count 3
az monitor autoscale rule create -g $RG --autoscale-name $AS \
   --condition "CpuUtilizationPercentage > 70 avg 5m"  --scale out 1 --cooldown 5
az monitor autoscale rule create -g $RG --autoscale-name $AS \
   --condition "CpuUtilizationPercentage < 30 avg 15m" --scale in 1  --cooldown 15

# Seasonal profile: Black Friday / Cyber Monday 2026 runs at 3.5x peak (97 tps) -> start at 5 of 6
az monitor autoscale profile create -g $RG --autoscale-name $AS -n black-friday-2026 \
   --min-count 5 --max-count 6 --count 5 --timezone "UTC" \
   --start 2026-11-26T00:00 --end 2026-12-01T23:59 --copy-rules default
```

G. Storage lifecycle and setup

G.1 storage/lifecycle-fraudmlstore.json

```
{
  "rules": [
    { "name": "raw-transactions", "enabled": true, "type": "Lifecycle",
      "definition": {
        "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["raw-transactions/"] },
        "actions": { "baseBlob": {
          "tierToCool":    { "daysAfterModificationGreaterThan": 30 },
          "tierToCold":    { "daysAfterModificationGreaterThan": 90 },
          "tierToArchive": { "daysAfterModificationGreaterThan": 400 },
          "delete":        { "daysAfterModificationGreaterThan": 730 } } } } },

    { "name": "features-online-snapshots", "enabled": true, "type": "Lifecycle",
      "definition": {
        "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["features/online-snapshots/"] },
        "actions": { "baseBlob": {
          "tierToCool": { "daysAfterModificationGreaterThan": 7 },
          "delete":     { "daysAfterModificationGreaterThan": 60 } } } } },

    { "name": "features-training-snapshots", "enabled": true, "type": "Lifecycle",
      "definition": {
        "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["features/training-snapshots/"] },
        "actions": { "baseBlob": {
          "tierToCool":    { "daysAfterModificationGreaterThan": 30 },
          "tierToCold":    { "daysAfterModificationGreaterThan": 180 },
          "tierToArchive": { "daysAfterModificationGreaterThan": 365 },
          "delete":        { "daysAfterModificationGreaterThan": 2557 } } } } },

    { "name": "experiments", "enabled": true, "type": "Lifecycle",
      "definition": {
        "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["experiments/"] },
        "actions": { "baseBlob": {
          "tierToCool": { "daysAfterModificationGreaterThan": 30 },
          "delete":     { "daysAfterModificationGreaterThan": 90 } } } } },

    { "name": "models-rejected", "enabled": true, "type": "Lifecycle",
      "definition": {
        "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["models/"],
                     "blobIndexMatch": [ { "name": "retention", "op": "==", "value": "rejected" } ] },
        "actions": { "baseBlob": { "delete": { "daysAfterModificationGreaterThan": 90 } } } } },

    { "name": "models-staging", "enabled": true, "type": "Lifecycle",
      "definition": {
        "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["models/"],
                     "blobIndexMatch": [ { "name": "retention", "op": "==", "value": "staging" } ] },
        "actions": { "baseBlob": {
          "tierToCool": { "daysAfterModificationGreaterThan": 30 },
          "delete":     { "daysAfterModificationGreaterThan": 730 } } } } },

    { "name": "models-production-working-copy", "enabled": true, "type": "Lifecycle",
      "definition": {
        "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["models/"],
                     "blobIndexMatch": [ { "name": "retention", "op": "==", "value": "production" } ] },
        "actions": { "baseBlob": {
          "tierToCool": { "daysAfterModificationGreaterThan": 180 },
          "delete":     { "daysAfterModificationGreaterThan": 730 } } } } },

    { "name": "telemetry", "enabled": true, "type": "Lifecycle",
      "definition": {
```

```
        "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["telemetry/"] },
        "actions": { "baseBlob": {
          "tierToCool": { "daysAfterModificationGreaterThan": 30 },
          "tierToCold": { "daysAfterModificationGreaterThan": 365 },
          "delete":     { "daysAfterModificationGreaterThan": 730 } } } },

    { "name": "old-versions-and-snapshots", "enabled": true, "type": "Lifecycle",
      "definition": {
        "filters": { "blobTypes": ["blockBlob"] },
        "actions": {
          "version":  { "delete": { "daysAfterCreationGreaterThan": 30 } },
          "snapshot": { "delete": { "daysAfterCreationGreaterThan": 30 } } } } }
  ]
}
```

G.2 storage/lifecycle-fraudmlaudit.json

```
{
  "rules": [
    { "name": "audit7y-predictions", "enabled": true, "type": "Lifecycle",
      "definition": {
        "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["audit-7y/predictions/"] },
        "actions": { "baseBlob": {
          "tierToCool":    { "daysAfterModificationGreaterThan": 30 },
          "tierToCold":    { "daysAfterModificationGreaterThan": 90 },
          "tierToArchive": { "daysAfterModificationGreaterThan": 365 },
          "delete":        { "daysAfterModificationGreaterThan": 2558 } } } } },

    { "name": "audit7y-records", "enabled": true, "type": "Lifecycle",
      "definition": {
        "filters": { "blobTypes": ["blockBlob"],
                     "prefixMatch": ["audit-7y/models/", "audit-7y/approvals/",
                                     "audit-7y/drift-decisions/", "audit-7y/rollbacks/"] },
        "actions": { "baseBlob": {
          "tierToCool":    { "daysAfterModificationGreaterThan": 30 },
          "tierToArchive": { "daysAfterModificationGreaterThan": 730 },
          "delete":        { "daysAfterModificationGreaterThan": 2558 } } } } },

    { "name": "audit2y-cicd", "enabled": true, "type": "Lifecycle",
      "definition": {
        "filters": { "blobTypes": ["blockBlob"], "prefixMatch": ["audit-2y/cicd/"] },
        "actions": { "baseBlob": {
          "tierToCool": { "daysAfterModificationGreaterThan": 1 },
          "delete":     { "daysAfterModificationGreaterThan": 731 } } } } }
  ]
}
```

G.3 infra/storage_setup.sh (Terraform equivalent in infra/)

```
#!/usr/bin/env bash
# Storage setup (normally in Terraform; CLI shown for readability)
set -euo pipefail
RG=fraud-ml-rg

# fraudmlstore (existing, LRS): soft delete 14 d, versioning, lifecycle, daily inventory
az storage account blob-service-properties update -g $RG -n fraudmlstore \
   --enable-delete-retention true --delete-retention-days 14 \
   --enable-container-delete-retention true --container-delete-retention-days 14 \
   --enable-versioning true
az storage account management-policy create -g $RG --account-name fraudmlstore \
   --policy @storage/lifecycle-fraudmlstore.json

# fraudmlaudit (new, GRS, private endpoint, Entra-only access, minimum TLS enforced)
```

```
az storage account create -g $RG -n fraudmlaudit -l eastus2 --sku Standard_GRS --kind StorageV2 \
   --allow-blob-public-access false --allow-shared-key-access false --min-tls-version TLS1_2 \
   --public-network-access Disabled
az storage container create --account-name fraudmlaudit -n audit-7y --auth-mode login
az storage container create --account-name fraudmlaudit -n audit-2y --auth-mode login

# Time-based retention (WORM). Unlocked for the first 30-day validation period, then locked.
az storage container immutability-policy create -g $RG --account-name fraudmlaudit -c audit-7y \
   --period 2557 --allow-protected-append-writes-all true      # ASA output commits blocks
az storage container immutability-policy create -g $RG --account-name fraudmlaudit -c audit-2y \
   --period 730
az storage account management-policy create -g $RG --account-name fraudmlaudit \
   --policy @storage/lifecycle-fraudmlaudit.json
# After validation (cannot be undone):
# ETAG=$(az storage container immutability-policy show -g $RG --account-name fraudmlaudit \
#        -c audit-7y --query etag -o tsv)
# az storage container immutability-policy lock -g $RG --account-name fraudmlaudit -c audit-7y \
#        --if-match "$ETAG"

# Event Hubs: 72 h retention covers a weekend under the 4-hour on-call SLA
az eventhubs eventhub update -g $RG --namespace-name fraud-events-ns -n transactions --retention-time 72
az eventhubs eventhub create -g $RG --namespace-name fraud-events-ns -n scoring-telemetry \
   --partition-count 8 --retention-time 72
az eventhubs namespace update -g $RG -n fraud-events-ns --capacity 2 \
   --enable-auto-inflate true --maximum-throughput-units 4

# Log Analytics: 90 d interactive, 2 y total for the MLOps tables
for T in DriftResults_CL FraudOps_CL PerfResults_CL AmlOnlineEndpointTrafficLog; do
  az monitor log-analytics workspace table update -g $RG --workspace-name fraud-ml-logs -n $T \
     --retention-time 90 --total-retention-time 730
done
```

H. Runbook command reference

H.1 CLI and KQL used by procedures R1–R7

```
# --- Endpoint state, traffic, release state (R1, R2, daily check) ---
az ml online-endpoint show -n fraud-scoring-prod -g fraud-ml-rg -w fraud-ml-workspace \
   --query "{state:provisioning_state, traffic:traffic, mirror:mirror_traffic,
             release:tags.release_state, rollback_target:tags.rollback_target}"
az ml online-deployment list -e fraud-scoring-prod -g fraud-ml-rg -w fraud-ml-workspace \
   --query "[].{slot:name, state:provisioning_state, instances:instance_count, model:model}" -o table

# --- R2 manual rollback: preferred (records audit trail) ---
gh workflow run ml-pipeline-rollback.yml -R fintech-corp/fraudshield-mlops \
   -f reason="INC-1234: p99 410 ms after release 3.7.0"
# --- R2 break-glass if GitHub is unavailable (then run the workflow later to write the record) ---
TARGET=$(az ml online-endpoint show -n fraud-scoring-prod --query tags.rollback_target -o tsv)
az ml online-endpoint update -n fraud-scoring-prod --traffic "$TARGET=100 <current-live>=0"

# --- Latency and errors per slot, 5-min bins (KQL, fraud-ml-logs) ---
AmlOnlineEndpointTrafficLog
| where TimeGenerated > ago(1h) and EndpointName == "fraud-scoring-prod"
| summarize requests = count(), p99_ms = percentile(TotalDurationMs, 99),
            error_rate = countif(ResponseCode >= 500) * 1.0 / count()
  by DeploymentName, bin(TimeGenerated, 5m)
| order by TimeGenerated desc

# --- Drift decisions and actions taken, last 24 h ---
DriftResults_CL
| where TimeGenerated > ago(24h) and Severity in ("Medium", "High", "Critical")
| project TimeGenerated, Signal, Feature, Value, Threshold, Severity, WindowsBreached, ActionTaken, AlertId
| order by TimeGenerated desc
```

```
# --- Predicted-fraud-rate ratio vs hour-of-week expectation (Incident #1 detector) ---
FraudOps_CL
| where TimeGenerated > ago(12h) and IsMirror == false
| summarize flags = sum(Flags), n = sum(N) by Hour = bin(WindowEnd, 1h), Deployment
| extend pfr = flags * 1.0 / n
| join kind=inner (PfrExpected_CL | project HourOfWeek, ExpectedPfr) on $left.Hour == $right.HourOfWeek
| extend ratio = pfr / ExpectedPfr

# --- Labeled performance trend (daily perf-monitor) ---
PerfResults_CL
| where TimeGenerated > ago(30d)
| project Cohort, ModelVersion, Recall, Auc, F1, RecallDropVsBaseline, LabelCompleteness

# --- Storage: tier moves and deletions done by the lifecycle policy (last 7 d) ---
StorageBlobLogs
| where TimeGenerated > ago(7d) and OperationName in ("SetBlobTier", "DeleteBlob")
| summarize ops = count()
  by OperationName, Container = tostring(split(ObjectKey, "/")[2]), bin(TimeGenerated, 1d)

# --- R5 feature store: Redis health ---
az redisenterprise show -n fraud-features -g fraud-ml-rg --query "{state:provisioningState, sku:sku.name}"

# --- R7 audit evidence: everything for one model version ---
az ml model show -n fraud-detection-xgboost -v <version> --query tags
az storage blob list --account-name fraudmlaudit -c audit-7y \
   --prefix "models/fraud-detection-xgboost/<semver>/" \
   --auth-mode login -o table
```

I. Configuration consistency matrix

Each value below appears in both the body and at least one configuration file. `check_config_consistency.py` (Appendix B.2) fails the build if the configuration files disagree with each other.

Parameter	Value	Body	Config
Endpoint and slots	`fraud-scoring-prod`; blue/green alternate	2, 4	B.6, B.7, F.1, F.2, H.1
Instance type and autoscale	DS3_v2; 3–6; CPU > 70% 5 min / < 30% 15 min; Black Friday minimum 5	2.7	F.2
Per-instance throughput floor	28 tps	2.7, 4.2	C.1 (G3)
Latency target	p99 ≤ 120 ms	2.6, 4.2	C.1 (G3, G11)
Abort rules	p99 > 300 ms over 3 min; errors > 0.5% with ≥ 5; PFR outside 0.5–2.0	4.3, 7.3	C.1 (abort)
Shadow	20% mirrored, 30 min, PFR 0.80–1.25, score PSI < 0.10	4.2	C.1 (G10), B.6
Canary	10%, ≥ 5,000 requests, ≤ 180 min, errors ≤ 0.1%	4.2	C.1 (G11), B.6
Standby	2 instances, 7 days	4.3	C.1, B.6
Offline gates	AUC ≥ 0.94; F1 ≥ 0.86; P ≥ 0.82; R ≥ 0.91; FPR ≤ 0.02; SHAP ≥ 0.85; fairness ≤ 0.05	3.5	C.1, A.1, B.4
Cost model	FN $127, FP $8	3.5, 5.4	C.1, A.3
Run contract	20 parameters, 16 metrics, 12 tags	3.2	A.1, A.3, B.4
Names	`fraud-detection-classification-v3`; `fraud-detection-xgboost`	3.1	A.1–A.4, B.4–B.7
Weekly retrain and window	Sunday 02:00 UTC; 90 days ending D-3	4.1, 5.6	B.1, B.3
Deploy window and freezes	Tue–Thu 13:00–17:00 UTC; Black Friday freeze 23 Nov–2 Dec 2026	4.2	D.2, B.6

Parameter	Value	Body	Config
Drift windows	1 h tumbling, persistence 2; ops 5 min	5.2, 5.3	D.1, E.1
Drift thresholds	PSI .10/.20/.25; KS .10/.15/.20; KL .05/.10/.125; PFR bands; unseen 0.5/1/3/5%; fallback 0.5/1/5%	5.3	D.1
Retrain guardrail	At most one drift retrain per 24 h	5.6	D.1
Event Hubs	16 and 8 partitions; 72 h; 2 TU auto-inflate to 4	2.5, 6.1	G.3
Stream Analytics	3 SU, 6 SU for seasonal events	2.4	E.1
Redis timeout; request timeout	25 ms; 1,000 ms server, 250 ms gateway	2.6	F.1
Raw retention	Cool 30, Cold 90, Archive 400, delete 730 days	6.1	G.1
Experiments retention	Cool 30, delete 90 days	3.3, 6.1	G.1, A.1
Model packages	rejected 90 d; staging and production 2 y; audit copy 7 y	3.4, 6.1	G.1, A.1, A.4
Audit retention	Predictions to Archive at 365 d; WORM 2,557 d; CI logs 730 d	6.1	G.2, G.3, B.8
Log Analytics	90 d interactive, 730 d total	6.1	G.3

J. Template review: corrections log

Defects found in the supplied templates and how this package resolves them.

#	Template, location	Issue	Resolution
1	GitHub Actions, §2–7	Six workflows chained by `workflow_run`; GitHub stops such chains after three levels, so Deploy never triggers	Orchestrator calling reusable workflows with `needs`: (B.1)
2	GitHub Actions, validate/promote	`download-artifact` without a run ID cannot read another workflow's artifact, and the path `mlflow_run_id.txt/mlflow_run_id.txt` is wrong	Values passed as job outputs
3	GitHub Actions, promote	Writes `stage=staging` but queries `Staging`, so verification always fails	One lowercase tag vocabulary (A.1)
4	GitHub Actions, promote/rollback	Version = Unix timestamp; "previous" found by sorting strings; several versions left tagged Production	Auto-increment versions; lifecycle and role tags; rollback target from the endpoint tag
5	GitHub Actions, deploy	Step output `decision` is never mapped to a job output, so the full-traffic job never runs	Gate logic in one job via `release_controller.py`
6	GitHub Actions, deploy	Deletes blue after promotion and never renames; the next run's traffic split names a deleted slot	Alternating slots; previous kept as warm standby
7	GitHub Actions and runbook, rollback	Rollback creates a new deployment, which takes minutes to provision, so < 5 min cannot be met	Traffic switch to the warm standby; redeploy only as a slow fallback
8	GitHub Actions, deploy	Fixed 30-minute canary: about 2,100 requests at average load, far fewer at night	Canary of ≥ 5,000 requests in a daytime window
9	GitHub Actions, all	`azure/login` with a stored service-principal JSON	OIDC federated credentials
10	GitHub Actions vs diagram	`ubuntu-latest` runners cannot reach private endpoints (the diagram already says self-hosted)	Self-hosted runners in the VNet for Azure jobs
11	GitHub Actions, deploy	5,000 ms request timeout against a 120 ms target; probe keys `initial_delay_seconds` are not the v2 schema names	1,000 ms server and 250 ms gateway timeouts; `initial_delay/period`
12	GitHub Actions, §9	No plan for 2-year CI/CD log retention; GitHub keeps logs at most 400 days	Nightly export to `audit-2y` (B.8)
13	MLflow, §4	Auto-promotion after 4 h without a block contradicts G8 and approval traceability	Explicit approval with prevent self-review

#	Template, location	Issue	Resolution
14	MLflow, §3 and §8	Checklist counts (15 parameters, 12 metrics) differ from the lists (20, 13); TensorFlow seed, epochs, batch size meaningless for XGBoost	Contract of 20 parameters, 16 metrics, 12 tags enforced as G7
15	MLflow, §7.2	Registration and stage transition inside training duplicate the gates; stage API deprecated	Training only logs; validate registers; lifecycle tags
16	MLflow, §4 G3	112 ms p99 measured offline excludes feature lookup and SHAP	G3 measured end to end on the candidate slot
17	MLflow, §5	Two-year archive of all experiments repeats Incident #3	Run artifacts deleted at 90 days after packages are copied out
18	Thresholds, §2.1–2.3	Five-minute windows with time-of-day features against an all-day baseline	Hourly, hour-matched baselines; time features excluded
19	Thresholds, §2.3	p-value bands impossible at n ≈ 8,333 (D = 0.05 already has p ≈ 10^{-17})	Effect-size thresholds calibrated against PSI
20	Thresholds, §2.2–2.3	KL 0.15 and KS 0.15 correspond to different drift sizes from PSI 0.25	All High thresholds ≈ 0.5σ (KL 0.125, KS 0.20)
21	Thresholds, §3.1	Treats latency as throughput (200 ms → 18,000 txn/h); 5 min at 300 ms priced at $2,667 instead of $267	Little's-law capacity plan; costs recomputed
22	Thresholds §3.3; runbook §1.3	Per-instance ≥ 28 tps and ≥ 450 requests/min as live alerts would fire every night (average per-instance load ≈ 3.9 tps)	Capacity tested in G3; CPU autoscale; traffic-versus-last-week alert
23	Thresholds, §6	Rollback on any drift during canary, though input drift is identical for both slots	Abort only on divergence between slot outputs
24	Diagram	Real-time scoring through Event Hubs and Stream Analytics cannot answer within 120 ms	Synchronous REST; Event Hubs is the asynchronous copy
25	Diagram	Cosmos DB telemetry with a 2-year TTL conflicts with 7-year retention of predictions and features	Immutable Blob audit account
26	Diagram	Raw 2.1 TB/month implies ~68 KB per transaction	Kept as a planning ceiling; validated in month 1 (A9)
27	Scenario A1	Capacity argued per partition; Standard tier is limited by throughput units	TU-based sizing; conclusion unchanged
28	Scenario, compliance	Unkeyed SHA-256 card hashes; PCI DSS v4.0 3.5.1.1 expects keyed hashes	HMAC-SHA-256 upstream (A8)
29	Runbook, §7.1	Endpoint logs queried from `AzureDiagnostics`	Resource-specific `AmlOnlineEndpointTrafficLog` table
30	Scenario infrastructure	Event Hubs retention of 24 h is shorter than a weekend under a 4-hour SLA	72 h retention

K. References

- Scenario Data Package v1.0 and the five supplied templates (MLflow configuration, GitHub Actions workflows, threshold decision, runbook, draw.io diagram), 30 July 2026.
- Cloudchipr, "Azure Blob Storage Pricing Breakdown: What You Need to Know", 6 February 2025, https://cloudchipr.com/blog/azure-blob-storage-pricing (per-GB tier prices and minimum retention periods used in Section 6.3).
- Microsoft Learn documentation for Azure Machine Learning managed online endpoints (safe rollout, mirrored traffic, autoscale), Azure Blob Storage lifecycle management and immutable storage, Event Hubs Capture and throughput units, and Stream Analytics windowing functions.
- GitHub Docs: reusing workflows, the `workflow_run` event, deployment environments, and OpenID Connect with Azure.
- PCI Security Standards Council, PCI DSS v4.0, Requirement 3.5.1.1 (keyed cryptographic hashes of PAN).